

Masters Degree Project

An LLM-Agent System for Interactive Data Analysis

Design, Implementation, and Performance Evaluation



Author: Nicolas Mahn
Supervisor: Rafael Martins
Semester: VT 2025
Course: 5DV50E
Subject: Computer Science

Abstract

Large Language Models (LLMs) are increasingly integrated into agent systems to tackle complex data analysis and code generation tasks. This thesis presents the design, implementation, and evaluation of a modular LLM-based agent system featuring diverse tools (sandboxed code execution, Retrieval Augmented Generation (RAG), short- and long-term memory, interactive Dash UI generation) and four distinct internal reasoning architectures: Simple, Planning, Reviewing, and Planning & Reviewing. The research systematically compares the performance of these agent configurations against raw LLMs and each other across a range of proprietary and open-source models. Evaluation employs benchmark metrics (Pass@1 on a subset of the BigCodeBench benchmark [1]) and a case study of multi-step interactive data analysis tasks.

The findings reveal that while agentic frameworks can enhance LLM performance, their efficacy is highly contingent on the specific architecture and the underlying model. The Reviewing Agent System demonstrated improvements for reasoning models like o4 mini (high) and Deepseek R1, suggesting the value of structured verification. Conversely, planning-based architectures proved inefficient and impractical for the evaluated tasks. Simpler agent structures were sometimes detrimental, particularly for smaller models like GPT 4.1 nano, which also exhibited high sensitivity to the amount of contextual information provided via RAG. The study underscores that there is no universally optimal RAG configuration (top-k); performance is model-specific, and excessive context can be counterproductive. Experiments with long-term memory also indicated model-dependent benefits. Assessments in the case study highlighted persistent challenges in the comprehensibility of AI-generated user interfaces and significant latency issues with more complex agent systems.

This research concludes that the utility of LLM-agents is not guaranteed by architectural complexity alone but arises from a careful alignment of model capabilities, agent design, and task requirements. It highlights the nuanced benefits of reviewing mechanisms, the critical need for model-specific RAG tuning, and the current limitations of planning-centric approaches, offering empirical insights for the development of more effective and practical LLM-based agent systems.

Keywords: Large Language Model (LLM), AI Agents, Retrieval Augmented Generation (RAG), Interactive Data Analysis, Code Generation, Agent Architectures, Tool Integration, Memory Systems, Business Intelligence (BI)

Contents

1 Introduction **1**
- 1.1 Background . . . 1
- 1.2 Problem Formulation . . . 2
- 1.3 Objectives . . . 2
- 1.4 Motivation & Target Group . . . 3
- 1.5 Limitations . . . 4
- 1.6 Outline . . . 5

2 Related Works **6**
- 2.1 Review of Existing Research . . . 6
- 2.2 Comparison of Approaches . . . 8
- 2.3 Identified Gaps and Positioning of this Work . . . 9

3 Methodology **10**
- 3.1 Design Science Research . . . 10
- 3.2 LLMs Selected for Evaluation . . . 11
- 3.3 Benchmark . . . 12
  - 3.3.1 Benchmark Dataset . . . 12
  - 3.3.2 Evaluation Metrics . . . 12
- 3.4 Case Study . . . 12
  - 3.4.1 Case Study Design . . . 13

4 Implementation **15**
- 4.1 Architecture . . . 16
- 4.2 Knowledge Base Management . . . 16
- 4.3 Sampling Temperature . . . 17
- 4.4 Tools . . . 18
  - 4.4.1 Responding to the User (`<response>`) . . . 18
  - 4.4.2 Updating the Short-Term Memory (`<short_memory>`) . . . 19
  - 4.4.3 Adding a Long-Term Memory (`<long_memory>`) . . . 19
  - 4.4.4 Query (`<query>`) . . . 19
  - 4.4.5 Document Analysis (`<document>`) . . . 19
  - 4.4.6 Code Execution (`<code>`) . . . 20
- 4.5 Context Window . . . 21
- 4.6 Agent Systems . . . 24
  - 4.6.1 Simple Agent System . . . 24
  - 4.6.2 Planning Agent System . . . 25
  - 4.6.3 Reviewing Agent System . . . 27
  - 4.6.4 Reviewing & Planning Agent System . . . 28
- 4.7 Frontend Interface . . . 30

5 Results **33**
- 5.1 Comparative Performance of LLMs and Agent Architectures . . . 33
- 5.2 Influence of Long-Term Memory with Ground Truth . . . 35
- 5.3 Effect of RAG Top-k Setting . . . 36
- 5.4 Case Study: System Evaluation Results . . . 39
  - 5.4.1 LLM Wrapper Performance . . . 39
  - 5.4.2 Simple Agent System Performance . . . 41

5.4.3 Reviewing Agent Systems Performance 43
5.4.4 Planning Agent Systems Performance 45

6 Discussion **46**
6.1 RQ1: Agent System Efficacy and Architectural Impact 46
6.2 RQ2: Impact of RAG Top-k for Coding Tutorials 47
6.3 RQ3: Augmenting Smaller Models 48
6.4 Additional Key Findings 48
6.5 Interpretation of Observations 49
6.6 Implications of the Study 51

7 Conclusion **53**

8 Future Works **54**

9 Web References **55**

References **56**

A Further Information about the Case Study **A**
A.1 Prompts used for the Case Study . . . . . A
A.2 Overarching Evaluation Criteria for Case Study Tasks . . . . . D

B Example Prompt and Simple Agent System Solution in Quantitative Benchmark **F**

1 Introduction

Recent advances in Large Language Models (LLMs) have opened up entirely new modes of interacting with data, software, and information. Rather than functioning solely as passive text generators, LLMs are now being embedded into agent systems—autonomous or semi-autonomous frameworks that allow models to reason, plan, execute actions (such as writing and running sandboxed code), and interact with tools like APIs, vector databases for Retrieval-Augmented Generation (RAG), and UI frameworks [2, 3, 4, 5]. These new capabilities have the potential to transform LLMs from conversational assistants into productive collaborators in real-world data science and software engineering workflows [6, 7].

This thesis presents the design, implementation (section 4), and evaluation (sections 3 and 5) of an LLM-based agent system. The system includes components for secure code execution (section 4.4.6), distinct short-term and long-term memory management (sections 4.4.2, 4.4.3), document-aware reasoning using Retrieval-Augmented Generation (RAG) from web sources (section 4.2), dynamic context window management (section 4.5), and interactive frontend generation using the python library Dash (section 4.7). Four distinct agent reasoning architectures (Simple, Planning, Reviewing, Planning & Reviewing, section 4.6) were implemented, differing in their internal processing loops, such as adding explicit planning or self-verification steps.

The system enables a model to iteratively reason through problems, plan actions, use tools to verify hypotheses or retrieve context, and generate interactive outputs like data dashboards. Unlike prior studies, this thesis includes a detailed Implementation (section 4) to ensure that the system design is transparent and reproducible.

While prior work has shown that LLMs augmented with tools, such as retrieval modules (RAG) [2], code interpreters [4], or multi-agent workflows [7, 8, 9] can outperform base models on specific tasks, these improvements typically focus on one capability at a time. In contrast, this thesis introduces a holistic system that integrates multiple tools and resources.

The performance of different agent architectures is compared both to each other and against raw LLMs, including both proprietary and open-source models (section 3.2).

1.1 Background

Traditional LLMs like ChatGPT are powerful zero-shot learners capable of solving a wide range of natural language tasks [10, 11]. However, their ability to complete complex or multi-step problems — such as generating and validating code, transforming datasets into interactive visualizations, or answering document-specific queries — is limited when they operate in a purely "stateless" or non-interactive environment. In this setting, models lack persistent memory, iterative feedback loops, and structured workflows, which prevents them from reliably coordinating multi-step reasoning or tool use [12].

In response to these limitations, recent frameworks like AutoGen [13], MetaGPT [14], and specialized data science agents such as Data Interpreter [6] and DS-Agent [7] have emerged (section 2.1). These systems explore multi-agent collaboration, structured workflows, or case-based reasoning to tackle domain-specific or tool-requiring tasks. While they demonstrate the potential of agent-based approaches, most focus on isolated capabilities or lack detailed, reproducible implementation descriptions.

This thesis builds on these ideas by combining several capabilities — structured reasoning patterns (planning, reviewing), multiple memory types (short-term, long-term semantic, RAG), sandboxed code execution, document analysis, and Dash UI generation —

into a single, unified agent system (section 4). This system is presented with a detailed Implementation section to ensure transparency and reproducibility. Its performance is benchmarked against raw LLMs and simpler architectures (sections 3 and 5) to evaluate the practical benefits and trade-offs of this integrated approach.

1.2 Problem Formulation

While LLM-agent systems are rapidly evolving and various architectures are being explored (as discussed in section 2), most prior studies isolate single capabilities (planning, reviewing, retrieval, or tool use) and provide limited implementation detail, hindering replication. This thesis addresses that gap by combining these capabilities into a unified, openly documented system (section 4) and by systematically evaluating how concrete design choices in this system affect performance on predefined tasks and metrics (sections 3, 5). The scope is explicitly limited to this artifact, a single-analyst case study for data exploration and dashboarding, and a targeted code-generation benchmark.

The core problem this thesis addresses is:

> In the implemented agent system, how do specific design choices—reasoning architecture (Simple, Planning, Reviewing, Planning & Reviewing), external tools (secure code execution, RAG), and memory mechanisms—affect performance on code-generation and data-transformation tasks relative to raw LLMs and simpler agents, and which choices drive productivity and accuracy under the study's metrics?

The research questions are scoped to this artifact:

RQ1 (Architecture choice and use-case selection): Within this implemented system, what is the effect of adding planning and/or reviewing loops on Pass@1, case-study rubric score, and observed latency, compared to a raw LLM wrapper and the Simple Agent architecture, and which configuration is preferable for data exploration and dashboard generation given these metrics?

RQ2 (Configuration levers): How do RAG top-k settings and long-term memory seeded with ground truth shift Pass@1 and case-study rubric scores in this system?

RQ3 (Model–architecture interaction): To what extent can structured architectures in this system compensate for smaller or less capable models relative to stronger models?

1.3 Objectives

This thesis aims to:

1. Guided by the Design Science Research (DSR) paradigm outlined by Hevner et al. [15], this thesis rigorously designs, implements, and documents a modular LLM-agent system. The artifact features four distinct reasoning architectures that support planning, sandboxed code execution, document retrieval via RAG, multiple memory types, and interactive Dash UI generation (Section 4). .

2. Benchmark the agent systems against raw LLMs using tasks involving code generation (section 3.3) and multi-step interactive data analysis leading to dashboard creation (section 3.4).

3. Measure the effectiveness of added agent components — different reasoning architectures (planning, reviewing), integrated tools (code execution, RAG query), and memory systems — on task performance, code correctness (Pass@1), output quality.

4. Compare agent system performance across multiple LLMs (including Deepseek R1, GPT 4.1 nano, Qwen 2.5 Coder, and Llama 4 Scout; see section 3.2) to analyze model-dependence and the interaction between model capability and agent architecture.

5. Discuss the applicability of such systems to real-world productivity tasks, compare the performance trade-offs of different agent architectures, and propose future extensions in this system.

1.4 Motivation & Target Group

Recent advances in LLMs have demonstrated their ability to perform a wide array of tasks, from text generation and summarization to answering factual questions with fluency. These models often excel in zero-shot or few-shot learning scenarios for tasks that are well-defined and relatively self-contained [11]. However, their efficacy can diminish when faced with more complex, multi-step problems that demand iterative reasoning, interaction with external tools, or maintaining state over extended interactions [16]. For instance, while an LLM might generate a simple script or answer a direct question about a dataset, the end-to-end process of interactive data analysis — which can involve iterative data querying, transformation, visualization generation (e.g., creating a Dash dashboard), and progressive refinement based on intermediate results — often exceeds the capabilities of raw LLMs operating in a purely "stateless" or non-interactive environment. The challenges in generating consistently reliable and high-quality solutions for complex visualization tasks, such as interactive dashboards, highlight these limitations [12].

This gap in handling complex pipelines effectively motivates a shift towards more structured approaches, such as LLM-based agent systems. These systems aim to augment LLMs by embedding them within frameworks that provide capabilities for planning, tool utilization (like sandboxed code execution for generating Python-based Dash applications and RAG for knowledge retrieval), distinct short-term and long-term memory management, and iterative execution. In this thesis, the artifact is openly documented and the detailed Implementation is treated as a reported result to support transparency and replication (section 4). Such architectures offer a pathway to transform LLMs from passive generators into assistive collaborators, capable of tackling sophisticated tasks like the step-by-step creation and refinement of interactive data dashboards [2, 14, 7, 6, 17]. The central motivation for this thesis, therefore, is to systematically design, implement, and evaluate different configurations of such an agent system to understand how their architectural choices impact performance and utility in these demanding scenarios under predefined metrics and observed latency.

With an increasing demand for automation in data analytics, software prototyping, and knowledge management, the potential for these enhanced LLM-agents to act as assistive co-pilots is increasing. From a scientific perspective, this work contributes to the ongoing evaluation of tool-augmented intelligence and benchmarks how specific agent architectures (section 4.6) and external tools shape agent behavior and output quality. From an industry viewpoint, such agents could enable faster data exploration, more efficient internal tool development (e.g., by reliably generating interactive Dash dashboards, section

4.7), or streamlined automatic report generation, potentially replacing hours of manual work with intelligent workflows. Furthermore, the integrated RAG system (section 4.2) allows agents to quickly find relevant sections within large document corpora or knowledge bases, accelerating information retrieval compared to manual searches. Because cost and time matter in practice, this thesis also reports observed latency and operator intervention to inform deployment choices. Evaluating these systems, comparing different agent types, and testing various LLM backends can help determine when agents should be trusted, how much autonomy they can handle, and what design principles lead to robust performance across diverse tasks.

This research is therefore primarily intended for individuals and teams interested in leveraging conversational AI for more effective and accessible data analysis:

Business Intelligence (BI) Professionals and Data Analysts: This thesis offers insights for these professionals by exploring how LLM-agents, equipped with capabilities like automated Dash dashboard generation and natural language querying, can lead to faster and more intuitive insight discoveries.

Business Users and Domain Experts: Non-technical users who have datasets they need to understand and analyze can benefit from the approaches discussed. This work demonstrates how agent systems can provide an intuitive interface for asking questions about data and receiving insights without requiring coding or specialized BI tool expertise when paired with lightweight operational support.

Developers Creating Data Analytics Tools: Engineers interested in the practical implementation of LLM-agents for data interaction will find detailed examples, including the integration of tools like code execution and UI generation (section 4.7), the design of agent reasoning flows (section 4.6), and the utilization of RAG for contextual data understanding (section 4.2).

Researchers in AI and Human-Computer Interaction: Academics studying the capabilities and limitations of LLM-agents for data analysis tasks, human-AI collaboration in analytics, and the design of effective natural language interfaces for complex data will find empirical data and analysis relevant to their work.

1.5 Limitations

This study has several limitations that open avenues for future research. The evaluation is conducted on a specific set of LLMs (section 3.2) and a custom-built agent framework; results may differ with other models or multi-agent systems like those reviewed in section 2. The RAG knowledge base is static and focused on coding tutorials; its impact might vary with dynamic or domain-specific knowledge sources. The case study (see section 3.4), while detailed, is based on the evaluation of a single researcher (the author), and did not involve broader live user studies, which are crucial for assessing real-world usability and user satisfaction. Interpretation is bounded to the implemented artifact and the specific design choices evaluated (architectures, tools, memory); the research questions are explicitly scoped to this implementation and its metrics. The detailed Implementation (section 4) is reported as part of the results to support transparency and replication, yet alternative implementations could yield different behavior.

Standards and tool interfaces evolved during the project timeline. In particular, the Model Context Protocol (MCP) [5] matured while this work was in progress; the artifact

does not adopt MCP-based tool integration. Future MCP-aligned refactoring could alter reliability, latency, and comparability.

Model quality is a moving target. Several newer releases (e.g., GPT-5, Kimi K2, Qwen 3, recent Groq and Anthropic tool-using models) appeared after the experiments and may reduce tool-calling errors. Results here may understate current-best performance on the same tasks.

Cost constrained the breadth of experiments and configuration sweeps. Running agent benchmarks incurs nontrivial API and compute charges; expanded sweeps over models, prompts, and hyperparameters were limited by budget. Exact spend was not logged, but public benchmark operators report substantial recurring costs [18].

Finally, this thesis deliberately focuses on a technical audience. While the findings of this research are relevant to decision-makers and could be translated into managerial recommendations, fully addressing both audiences within a single document would compromise the depth and clarity of the analysis. Providing detailed architectural descriptions, implementation steps, and reproducible evaluation protocols already requires significant space and precision. In contrast, managerial communication would necessitate a different framing, such as simplified models, case-based examples, and business-oriented implications, ideally supported by live user studies and organizational data. Given the scope and length constraints of a master's thesis, prioritizing technical communication was necessary. However, the insights presented here form a foundation that future work could reframe into actionable guidance for organizations considering the adoption of LLM-agent systems in business intelligence workflows.

1.6 Outline

Section 1 introduces the research area of LLM-based agent systems, outlines the problem formulation, research questions, motivation, objectives, and scope of this work. Section 2 provides a review of existing research in natural language interfaces, LLM-agent frameworks, Retrieval-Augmented Generation, and code generation, comparing different approaches and positioning the current work within this landscape. Section 3 details the methodology employed for evaluating the implemented agent system, including the selection of LLMs, the benchmark setup using BigCodeBench prompts, and the design of a case study with data analysis tasks. Section 4 describes the architecture and components of the developed LLM-agent system, including its various reasoning architectures (Simple, Planning, Reviewing, Planning & Reviewing), toolset (code execution, RAG, memory management), context window management, and frontend interface. Section 5 presents the findings from the benchmark, analyzing the performance of different LLMs within various agent configurations, the impact of long-term memory, and the effects of RAG top-k settings. This section also includes a detailed case study of agent behavior on multi-step tasks. Section 6 interprets these findings in the context of the research questions, discusses their implications, compares them with existing literature, and acknowledges the limitations of the study. Finally, section 7 summarizes the key achievements and contributions of the thesis, reiterates the significance and generalizability of the results, and section 8 proposes directions for future research and potential improvements to the agent system.

2 Related Works

The development of Natural Language Interfaces (NLIs) for data interaction and the broader field of autonomous agents powered by LLMs have seen rapid advancements. Research in these areas aims to bridge the gap between complex data systems and users, often leveraging LLMs for planning, tool use, and interaction. This work on LLM-agents for data analysis builds upon these existing efforts but introduces distinct agent architectures and a specific tool interaction mechanism. A central objective of this thesis is to combine several agentic ideas—planning, reviewing, RAG, multiple memories, code execution, and UI generation—*simultaneously* and evaluate whether current LLMs can handle them in one system. This section reviews relevant research, compares different approaches, and positions this work within the current landscape.

2.1 Review of Existing Research

Several lines of research inform the development of agentic LLM systems.

NLIs for Specific Platforms. Tong et al. (2024) [19] present a Visualization-oriented Natural Language Interface (V-NLI) for Grafana dashboards. They use an LLM within a LangChain agent framework [20] to interpret user requests and select appropriate tools, but delegate the generation of Grafana-specific JSON configurations to a rule-based toolkit, highlighting challenges in LLM code generation for specialized formats.

Conceptual Frameworks for AI Integration. Bühler et al. (2024) [17] discuss the "AI-in-the-loop" paradigm for biomedical visual analytics (VA), emphasizing human agency while using AI as a collaborative tool. They outline potential AI roles in the VA workflow, such as *Data2Code*, *Task2Query*, and *Task2Interact*, providing a conceptual basis for AI-assisted data analysis, particularly in sensitive domains requiring transparency and reliability. Similarly, Tu et al. (2024) [21] argue that LLMs are shifting the role of data scientists towards managing AI-driven analyses, necessitating skills like critical thinking and strategic planning.

Multi-Agent Frameworks. Several frameworks explore collaboration between multiple LLM-agents. Wu et al. (2023) [13] introduce AutoGen, enabling multi-agent conversations through customizable, "conversable" agents and flexible "conversation programming." AutoGen also offers RAG agents (RetrieveChat) that integrate retrieval into the agent loop [13]. Hong et al. (2023) [14] present MetaGPT, which encodes human Standardized Operating Procedures (SOPs) into agents with predefined roles (e.g., Product Manager, Engineer) that collaborate by producing standardized outputs, mimicking software development teams. Li et al. (2023) [22] propose CAMEL, a framework using role-playing where an "AI user" agent provides instructions and a task-specific "AI assistant" agent executes them, guided by inception prompting to maintain roles and task focus. Liu et al. (2024) [23] propose DyLAN, featuring dynamic team optimization where agents are selected based on contribution scores and collaboration occurs in Temporal Feed-Forward Networks. Talebirad and Nadiri (2023) [8] offer a general conceptual framework for multi-agent collaboration defining agents (IGAs) by their LLM, role, state, and capabilities, allowing for dynamic interactions. Shen et al. (2024) [24] propose α-UMi, a multi-LLM framework decomposing tool-use capabilities into specialized planner, caller, and summarizer roles.

Agent Libraries and Tooling. Liu et al. (2024) [3] introduce AgentLite, a lightweight library focused on simplifying the prototyping and evaluation of different agent reasoning strategies and architectures, providing foundational building blocks rather than specific complex frameworks.

Retrieval-Augmented Generation (RAG) and Knowledge Bases. Augmenting LLMs with external knowledge is crucial for grounding responses and reducing hallucination. The general approach, often termed RAG [2], involves retrieving relevant information from an external corpus and providing it as context to the LLM during generation. Xue et al. (2024) [25] present DB-GPT, a system focused on enabling natural language interaction with traditional databases, emphasizing privacy through the use of private LLMs. DB-GPT features a multi-source RAG system incorporating vector, keyword, and graph indexing for knowledge retrieval from documents like PDFs and web pages, and includes fine-tuned models for Text-to-SQL generation. Sasaki et al. (2024) [26] explore dynamically integrating external knowledge bases with the Mistral LLM via a middleware layer that determines the need for external information, retrieves it, preprocesses it, and formats it for the LLM input, aiming to improve contextual understanding and factuality without altering the base model. Zong et al. (2024) [27] introduce Triad, a framework using a multi-role LLM-based agent (Generalist, Decision-Maker, Advisor) to handle the distinct phases of Knowledge Base Question Answering (KBQA), including question parsing, URI linking, query construction, and answer generation, without task-specific model training. Mahn and Abbas (2025) [28] present VotingAid, a RAG system using LLMs to compare political party manifestos based on user-defined concerns, offering a dynamic alternative to traditional Voting Advice Applications (VAAs). Within multi-agent libraries, AutoGen exposes ready-to-use RAG pipelines that couple retrieval with agent coordination [13]. This work (section 4.2) employs a RAG built from scraped web documents, using semantic vector similarity (via OpenAI's `text-embedding-ada-002`) for retrieval to provide context for data analysis tasks.

Data Science Agent Frameworks. Hong et al. (2024) [6] propose Data Interpreter, using Hierarchical Graph Modeling to decompose data science problems into task and action graphs, enabling dynamic workflow refinement based on intermediate results. Guo et al. (2024) [7] introduce DS-Agent, which applies Case-Based Reasoning (CBR) by retrieving and adapting solutions from past similar problems (initially from Kaggle, then self-generated) to guide planning and code generation.

Code Generation Aspects. Wang et al. (2024) [4] propose CodeAct, using executable Python code as the unified action format for agents, leveraging LLMs' code pre-training and enabling complex logic, data flow, and self-debugging via interpreter feedback. Wen et al. (2024) [29] introduce GIFT4CODE, an instruction fine-tuning method using I/O specifications derived from program execution (variable types, concrete examples, NL summaries) to ground code generation and reduce ambiguity. Pan et al. (2025) [30] focus specifically on code efficiency, proposing E-code which uses a multi-encoder architecture to optimize functionally correct but inefficient code. Improta et al. (2025) [31] investigate the robustness of AI code generation (specifically for offensive security code) to variations in natural language prompts, proposing data augmentation techniques to mitigate sensitivity.

2.2 Comparison of Approaches

The reviewed literature presents a diverse landscape of approaches for building and enhancing LLM-based agents. Key dimensions of comparison include agent architecture, collaboration mechanisms, action representation, learning/optimization strategies, and application focus.

Architecture. Approaches range from single-agent systems enhanced with specific techniques (e.g., GIFT4CODE [29], E-code [30], Improta et al. [31]) to complex multi-agent frameworks. Multi-agent systems differ in their structure and coordination: AutoGen [13] uses flexible "conversation programming," MetaGPT [14] employs rigid SOP-based roles, CAMEL [22] uses structured role-playing, DyLAN [23] features dynamic team optimization, α-UMi [24] decomposes internal capabilities using multiple LLMs, while Talebirad & Nadiri [8] provide a general conceptual model. Frameworks like Data Interpreter [6] and DS-Agent [7] propose architectures specifically tailored for data science workflows, using graph modeling and case-based reasoning, respectively. Libraries like AgentLite [3] aim to provide building blocks for various architectures. This LLM-agent system (section 4) utilizes an agent architecture but explores variations in internal reasoning patterns (Simple, Planning, Reviewing, Planning & Reviewing).

Action Representation and Tool Use A significant distinction lies in how agents perform actions, particularly tool invocation. Many systems, including this one (section 4.4), rely on predefined tool descriptions and specific invocation formats (e.g., JSON, XML tags). Tong et al. [19] use an agent to select tools but delegate specialized format generation. DB-GPT [25] and Triad [27] focus on database query generation (SQL/SPARQL). CodeAct [4] proposes using executable Python code itself as the action, allowing native composition and leveraging interpreter feedback. GIFT4CODE [29] focuses on grounding code generation using I/O specifications to improve reliability.

Knowledge and Memory Systems employ various methods to access information beyond the LLM's internal parameters. RAG is a common technique, used in this system (section 4.2), DB-GPT [25], Sasaki et al. [26], Triad [27] (implicitly via KB interaction), and VotingAid [28]. Implementations differ in knowledge source (web documents vs. databases vs. KBs vs. political manifestos vs. semi-structured knowledge bases) and retrieval/integration mechanisms (vector similarity vs. keyword/graph indexing vs. middleware vs. SPARQL vs. hybrid). DS-Agent [7] uses Case-Based Reasoning (CBR) to retrieve past problem-solving experiences. This system also incorporates distinct short-term and long-term semantic memory mechanisms (sections 4.4.2, 4.4.3).

Application Focus Research spans general agent capabilities, specific domains like software development (MetaGPT [14]), data science (Data Interpreter [6], DS-Agent [7]), database interaction (DB-GPT [25]), Knowledge Base Question Answering (KBQA) (Triad [27]), visualization (Tong et al. [19], Bühler et al. [17]), code efficiency (E-code [30]), code robustness (Improta et al. [31]). This work focuses on enabling general data analysis for users interacting with uploaded datasets via natural language.

2.3 Identified Gaps and Positioning of this Work

Firstly, much work on agent architecture focuses on multi-agent collaboration patterns (e.g., AutoGen, MetaGPT, CAMEL, DyLAN) or specific workflow structures for domains like data science (Data Interpreter) [13, 14, 22, 23, 6]. There is less emphasis on systematically exploring different internal reasoning architectures and tool-availability within an agent framework designed for a specific interaction paradigm, such as natural language data analysis. Even when RAG is supported (e.g., AutoGen), prior studies typically evaluate it as a single capability rather than in concert with multiple agentic mechanisms under one artifact and metric suite.

Secondly, while RAG is used to provide external knowledge (as in this system, section 4.2), and CBR (long-term memory, see section 4.4.3) is used to leverage past experiences (DS-Agent), the interplay between different memory types (short-term, long-term semantic memory, RAG sourced from web documents) and structured reasoning patterns (planning, reviewing) within an agent system focused on user-driven data analysis needs further investigation [25, 26, 27, 28, 32, 7].

This work positions itself to address these aspects. The LLM-agent system presented in this thesis (section 4) focuses on enabling users to perform data analysis through natural language interaction with uploaded datasets. Its key contributions relative to the reviewed literature are:

Exploration of Agent Reasoning Architectures: This project introduces and evaluates four distinct agent types (Simple, Planning, Reviewing, Planning & Reviewing - section 4.6) within a unified framework. These types represent different internal reasoning and execution loops (e.g., explicit planning, self-verification), allowing for a comparative study of how these architectural choices impact performance in data analysis tasks, contrasting with frameworks focused primarily on inter-agent communication or predefined roles.

Tool Invocation: The agent system presented in this thesis utilizes a specific set of tools invoked via XML tags (section 4.4), including a sandboxed code execution environment. This provides a structured yet flexible mechanism for action representation, differing from both free-form text/JSON calls and the direct executable code approach of CodeAct [4].

Integration of Multiple Knowledge/Memory Sources: The system integrates prompt context, short-term memory (section 4.4.2), long-term semantic memory (section 4.4.3), and a RAG database sourced from web documents (section 4.2), managed through a dynamic context prioritization mechanism (section 4.5), providing a rich information environment for the agent's reasoning process.

Combined-Capability Evaluation: Unlike works that test planning, reviewing, or retrieval in isolation, this thesis evaluates whether current LLMs can *simultaneously* coordinate planning/reviewing with RAG, memory, code execution, and UI generation inside one implemented system, reporting both performance metrics and observed latency.

Focus on General Data Analysis: Unlike systems tailored for specific platforms [19] or complex software development [14], this system aims to provide a general-purpose tool for users to interact with and analyze their own datasets using natural language.

3 Methodology

As presented in this thesis, a novel custom AI agent framework (detailed in section 4) was developed. This framework aims to enhance the capabilities of LLMs beyond their raw performance, particularly for complex tasks in dataset analysis, programming, and data science, by providing a unified system with structured reasoning, diverse memory systems, and integrated tools (as outlined in section 1). This section details the methodology employed to evaluate this framework. The primary step in this evaluation is to assess the added value of the agent's specific architectural designs (section 4.6) and its comprehensive toolset (section 4.4) in comparison to the baseline capabilities of unassisted LLMs. Subsequently, the second step involves a comparative analysis of the performance of the different agent architectures implemented—namely Simple, Planning, Reviewing, and Planning & Reviewing—both against each other and against the raw LLM benchmarks. The methodology is explicitly designed to answer **RQ1–RQ3**: architecture impacts and preferred configurations (**RQ1**), configuration levers such as RAG top-k and long-term memory (**RQ2**), and model–architecture interactions, i.e., whether structure can compensate for weaker models (**RQ3**).

To achieve these goals, the evaluation is split into a benchmark and a case study. The benchmark measures code generation capabilities, while the case study explores performance on more open-ended, multi-step data analysis tasks requiring interaction with the user interface. Concretely, the *benchmark* supplies controlled, repeatable measurements for Pass@1 under architectural and configuration variations (addressing **RQ1**, **RQ2**, and **RQ3**); the *case study* in Section 3.4 adds ecological validity by scoring interactive, user-facing artifacts with a rubric and observing latency, thereby complementing **RQ1** and **RQ3**.

3.1 Design Science Research

This thesis is explicitly framed within the Design Science Research (DSR) paradigm as articulated by Hevner et al. [15]. Following this framework ensures that the work not only produces a novel artifact but also rigorously evaluates and communicates its contributions in alignment with established guidelines.

The first guideline, **Design as an Artifact**, is met through the development of a modular LLM-agent system that integrates sandboxed code execution, Retrieval-Augmented Generation (RAG), multiple memory mechanisms, and an interactive Dash UI. Four distinct internal reasoning architectures (Simple, Planning, Reviewing, and Planning & Reviewing) are implemented, forming the core artifact of this research.

The second guideline, **Problem Relevance**, is addressed by targeting a pressing challenge in modern data science and business intelligence: current LLMs struggle to manage multi-step, tool-intensive tasks such as interactive data analysis and dashboard generation. The problem formulation and four guiding research questions presented in section 1.2 directly reflect this relevance to both research and practice.

The third guideline, **Design Evaluation**, is realized through a two-stage evaluation strategy. A quantitative benchmark using a subset of BigCodeBench measures Pass@1 code correctness, while a qualitative case study examines multi-step, realistic workflows for usability and performance. Together, these methods rigorously test the artifact's utility and efficacy.

The fourth guideline, **Research Contributions**, is fulfilled through three distinct outputs: (1) the implemented LLM-agent framework itself, (2) empirical insights into how agent architecture and reasoning strategies affect performance, and (3) methodological guidance for future evaluations of complex AI-agent systems. These contributions are synthesized and contextualized in section 6 and 8.

The fifth guideline, **Research Rigor**, is ensured by grounding the artifact's design and evaluation methods in existing literature, as reviewed in section 2, and by applying systematic experimental protocols and transparent performance metrics throughout the study.

The sixth guideline, **Design as a Search Process**, is reflected in the iterative exploration of design alternatives, including different reasoning architectures, RAG top-k configurations, and LLM backends. This iterative approach provides a structured search for effective solutions under varying conditions.

Finally, the seventh guideline, **Communication of Research**, is addressed by presenting this work in a form that enables replication and extension by the research community. Technical readers are provided with detailed architectural descriptions, open implementation details, and transparent evaluation protocols. While this thesis primarily targets a technical audience, the results are framed in a way that could later be translated into managerial insights for organizations considering the adoption of LLM-agent systems.

By explicitly aligning this thesis with the DSR paradigm, each step, from problem formulation to evaluation and communication, contributes both to academic understanding and to practical advancements in the field of AI-driven data analysis.

3.2 LLMs Selected for Evaluation

The models chosen for this evaluation span major proprietary providers and the open-source domain to ensure diverse capabilities and cost profiles. The goal was to analyze how agent performance is influenced by varying model characteristics such as reasoning ability, context window size, code generation proficiency, and parameter count. This diversity is necessary to probe **RQ3**, enabling comparisons of weaker vs. stronger models under matched architectural conditions.

The primary models chosen for in-depth comparative analysis in the benchmark and case study were selected for their distinct features. Deepseek R1 was included as it represents high-end open-source reasoning capabilities [33]. From OpenAI, GPT 4.1 mini was a core model due to its significant capabilities and its 1M token size limit[11]. Complementing this, GPT 4.1 nano, also from OpenAI and featuring a 1M token size limit, served as a cost-effective option with streamlined processing capabilities [11]. For dedicated code-generation tasks, Qwen 2.5 Coder 32B was incorporated as it leads the open big code leaderboard [34]. Lastly, Meta's Llama 4 Scout was chosen, enabling the study of agent performance with extensive contextual information.

Beyond these core models other LLMs were incorporated to broaden the evaluation and ensure a more comprehensive assessment. These included OpenAI's o4-mini, selected based on strong performance metrics and thinking capabilities, with it set to 'high reasoning'. Further diversity was introduced with Deepseek V3, offering strong non-reasoning performance. Additional models from Meta's Llama series, such as Llama 4 Maverick, chosen for its high intelligence within the Llama family, and Llama 3.3 70B,

were also part of the broader test suite. This diverse set allows for a comprehensive analysis of how different model characteristics influence agent performance within the framework [11].

3.3 Benchmark

The benchmark aims to evaluate the code generation capabilities of the different agent architectures and raw LLMs on a standardized set of data science programming tasks. This focuses specifically on the `<code>` tool's effectiveness (section 4.4.6) when guided by different agent reasoning processes. These controlled measurements provide the primary evidence for **RQ1** (architectural impact on Pass@1) and **RQ2** (effects of top-k and long-term memory).

3.3.1 Benchmark Dataset

Tasks from the BigCodeBench benchmark [1] are utilized, specifically selecting Python-based data science problems, to benchmark the agent systems. This benchmark was chosen because it provides ground truth test cases for automatic evaluation of code correctness and covers a range of common data science operations (e.g., using Pandas, NumPy, Scikit-learn). A subset of 100 tasks appropriate for the agent's capabilities were selected.

Each agent type and raw LLM is presented with the natural language description of the BigCodeBench task. The agent/LLM is expected to generate Python code within the `<code>` tag (for agents) or directly (for raw LLMs) to solve the task. The generated code is executed against the benchmark's built-in test cases. Runs are performed with the agent's long-term memory retrieval disabled (section 4.4.3) to assess baseline code generation performance without learning from prior attempts on the benchmark. Short-term memory and RAG remain active. Some test focuses on specific features such as the ideal top-k value. Top-k refers to the number of documents (k) retrieved through RAG. One run is designed to test the long-term memory function of the agents, as the long-term memory is turned off for all other tests (including the case study, see section 3.4). For this test, the agent system is initially prompted with the ground truth, using the GPT 4.1 mini model. The agent is asked to store this information in its long-term memory. The benchmark tasks are then run again with long-term memory enabled to test if the agent can leverage the stored correct solution to improve performance on subsequent runs. These ablations directly target **RQ2** by isolating configuration effects under otherwise identical conditions.

3.3.2 Evaluation Metrics

The metric used for evaluation is **Pass@1**. It indicates whether the first code solution generated by the agent/LLM passes all the ground truth test cases for a given problem [35]. Execution accuracy (percentage of tasks for which functionally correct code was generated) is calculated across the selected benchmark subset for each agent system type, raw LLM, top-k setting, and memory condition.

3.4 Case Study

The case study method was selected to investigate how the implemented agent-based system performs in realistic, multi-step data analysis tasks. Unlike synthetic benchmarks that

focus solely on accuracy or single-step problem-solving, this case study provides contextual insights into how different agent architectures behave when confronted with iterative, tool-based workflows and interactive visualization requirements. This approach directly addresses the complexity of our research problem, where understanding *how* and *why* certain architectures succeed or fail is as important as measuring whether they produce correct outputs [36, 37].

This design supports answering **RQ1** by comparing the effects of planning and reviewing loops on both functional metrics (Pass@1) and qualitative performance indicators like usability and clarity. It also provides a context-rich environment to explore **RQ3**, by showing whether structured agent architectures can compensate for weaker models when executing realistic tasks.

In addition to measuring correctness, this case study emphasizes non-functional characteristics such as clarity, usability, and presentation quality of outputs, which are crucial for real-world adoption. This complements the quantitative benchmark results by providing a holistic view of how the system performs as a user-facing solution.

3.4.1 Case Study Design

To implement this approach, four tasks were designed, each representing a distinct and realistic data analysis challenge. These tasks progressively increase in complexity, ranging from simple exploratory visualizations to multi-source data integration, thereby reflecting the diverse conditions under which such agent systems might be deployed.

The experiment comprises of 4 tasks with 3.5 prompts per task. For each task, all agent system types, and the LLM Wrapper are given a prompt with a specific goal. Each agent type operates within the framework described in section 4, utilizing the defined tools (section 4.4), knowledge base (section 4.2), and context management (section 4.5). Agents interact with the user through the simulated chat interface, receiving prompts and producing responses or artifacts (code, dashboards). The `<code>` tool executes generated Python code in the sandboxed environment. If the code generates a Dash application, it is deployed, and the agent receives a multimodal report on the rendered UI (section 4.4.6). To ensure reproducibility and isolate the effect of the agent architecture, the long-term memory retrieval function (section 4.4.3) is disabled during these specific tests, preventing variability from prior interactions. Short-term memory and RAG remained active.

The LLM Wrappers (raw LLMs) are interacted with directly via their APIs. To emulate the agent system's automatic document analysis upon upload (section 4.4.5), the raw LLMs are provided with the same initial document summary in their first prompt. Code generated by raw LLMs is executed externally in an identical sandboxed environment. All agent types and the raw LLMs are given a dataset to analyze per task and are prompted with the same equivalent instruction set to evaluate their responses.

Each task has their own objective, with the goal of inferring the specific capabilities of the agents.

Task 1: Data Exploration & Neural Networks: The first task revolves around the analysis of an employee dataset [38]. It is designed to test the agent's ability to produce simple graphs and, in its second half, to create small neural networks and utilize SHAP (SHapley Additive exPlanations, a method to explain individual predictions of machine learning models by attributing importance scores to input features [39]) for creating an interactive Dash frontend. The exact prompts used can be found in the Appendix A.1.

Key evaluation criteria for Task 1 include: successful generation of visualizations

comparing education and payment tier; ability to update the dashboard to incorporate experience analysis; correct implementation of a neural network for attrition prediction; functional integration and interpretation of SHAP values to explain predictions; and the overall usability and clarity of the generated Dash frontends.

Task 2: Dimensionality Reduction & Clustering: The second task focuses on a simulated dataset that explores the habits and performance of students [40]. Which is intended to test the agents ability to create a dimensionality reduced graphs. Further the agents are then required to create a complex dashboards, that enable varied selections between using dataset derived segmentation and clustering using clustering algorithms. This tests not only the ability to cluster data but also to create a dashboard where a user can make a complex selection. The exact prompts used can be found in the Appendix A.1.

Key evaluation criteria for Task 2 include: correct application of user-selectable dimensionality reduction techniques (PCA, t-SNE, UMAP); ability to perform clustering using user-selectable algorithms (K-Means, DBSCAN, Agglomerative Clustering) on original features; and successful creation of an interactive 2D scatter plot where point coloring can be dynamically changed based on cluster labels or other original dataset attributes.

Task 3: Data Preprocessing: The third task attempts to test the data engineering skills of the agent(s), using a unclean smartwatch dataset [41]. This tests the agents ability to clean data. The dataset simulates health-related outputs from a smartwatch, mimicking real-world issues in data collection, making it perfect for applying data preprocessing techniques such as handling missing values, outliers, duplicates, and inconsistencies. The exact prompts used can be found in the Appendix A.1.

Key evaluation criteria for Task 3 include: effective handling of missing values, correction of data types, standardization of categorical values, identification and handling of outliers; successful creation of a cleaned data file; and generation of a dashboard visualizing quantified data quality issues per column and total duplicate rows.

Task 4: Handling multiple Datasets: In the last task the agents attempt to complete the slightly adapted GAStech mini challenge from 2021 [42]. This tests the agents' ability to work on bigger projects, with suboptimal data types (like images). The files were slightly altered and converted to utf-8 (as the original types are not supported by the document analysis tool). The street data was converted to a geojson, as the initial file format was also not supported. This task is presumably not solvable by any agent system as it involves the mapping of an image on to the street data. Parts of this task aim to define the upper limits of the given agent system in solving complex programming problems. The exact prompts used can be found in the Appendix A.1.

Key evaluation criteria for Task 4 include: identification of popular locations and anomalies from credit/loyalty card data and visualization in a dashboard; ability to concatenate datasets with a complex relation; appropriate updates to the dashboard with the agents findings.

4 Implementation

This section describes the implementation of the LLM-agent systems designed to enable users to upload datasets and interact with the agent through natural language. The system empowers users — regardless of technical background — to analyze data, identify patterns, and extract insights in a semi-autonomous manner.

To explore differences in agent behavior, four distinct LLM-agent system variants were created, each defined by unique sets of guiding instructions. Unlike conventional LLMs, these agents utilize additional contextual information and specialized tools to interpret, manipulate, and respond to uploaded data. The following subsections detail these components.

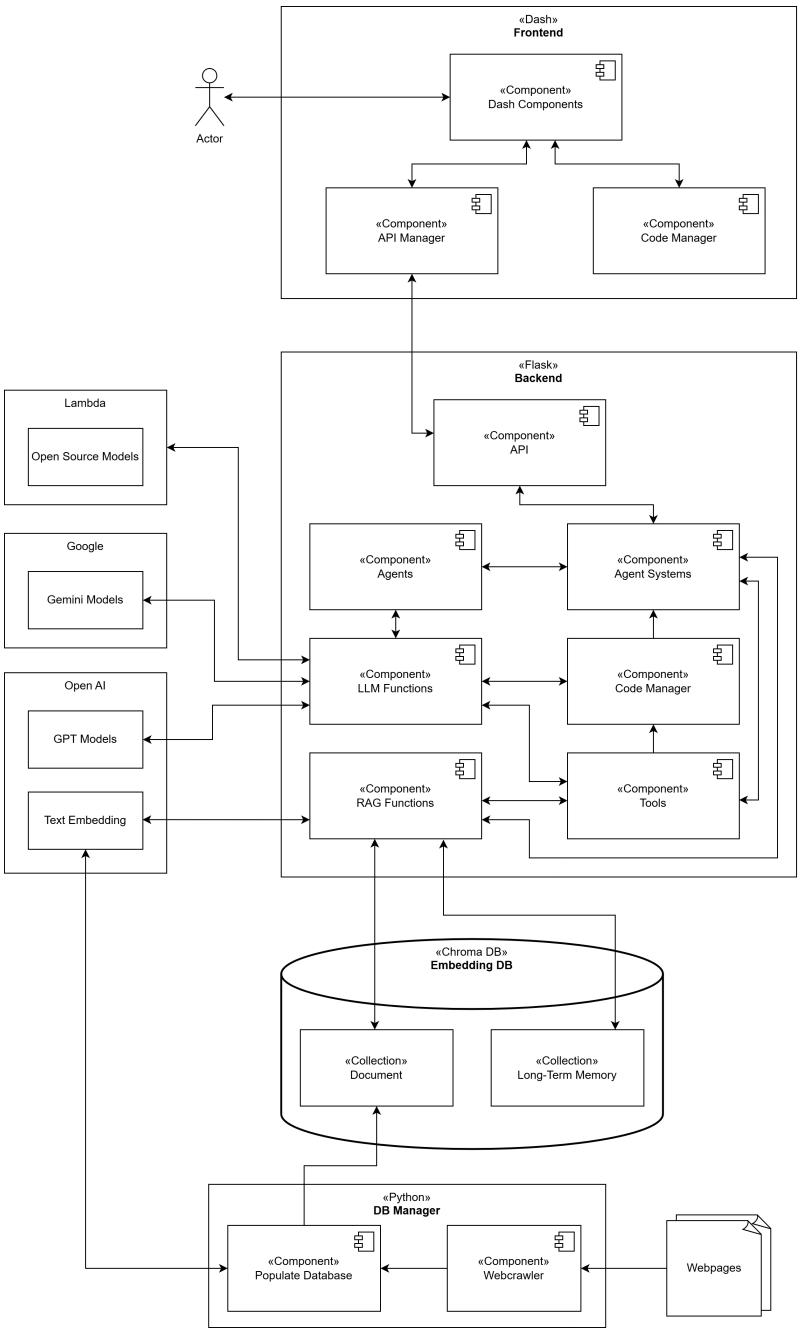


Figure 4.1: Agent System Architecture

This thesis follows a design-science approach: the implemented agent system serves both as a research artifact and as an experimental testbed. This thesis is organized to first describe the design and then evaluate it against the research questions.

4.1 Architecture

The system employs a modular architecture designed to facilitate user interaction with specialized LLM-agents for data analysis. As depicted in figure 4.1, the core components include a web-based frontend interface, a backend service layer managing state and communication, the LLM-agent reasoning engine, a RAG database for external knowledge integration, a suite of specialized tools for agent actions, and a sandboxed environment for secure code execution.

User interactions initiated via the Dash-based frontend (section 4.7) are transmitted through a REST API to the backend manager. This layer orchestrates the workflow, invoking the selected LLM-agent system variant (section 4.6), which in turn invokes the LLM-agent with the appropriate context (section 4.5). The agent processes user prompts by leveraging its internal knowledge, the managed context window, the external RAG (section 4.2), and a defined set of tools (section 4.4). The code execution tool is dispatched to isolated Docker containers, ensuring security and resource management. The results are relayed back to the agent, which formulates a response or undertakes further actions. This decoupled design promotes scalability and maintainability by separating concerns between user interface, agent logic, knowledge management, and execution environments.

Summary of Flow. In essence, user (`Actor`) requests traverse from the `Frontend` to the `Backend`. The `Backend`'s `Agent` component orchestrates task execution by leveraging external `LLM Functions`, internal knowledge via `RAG Functions` accessing the `Embedding DB`, and specific `Tools`, including sandboxed `Code-Manager` execution. The `Embedding DB` itself is populated independently by the `DB Manager` component, which crawls and processes external `Webpages`. When the agents are done the `Backend` informs the `Frontend` about the update using a socket.io event. The arrows throughout figure 4.1 trace these intricate flows of data and control signals, highlighting the system's modular and interconnected nature.

4.2 Knowledge Base Management

A core component of the agent-based system is its external knowledge base (RAG). In addition to the agent's memory and prompt context, this RAG enables the system to access and reason over a large corpus of curated external information, aiming to ground responses and reduce hallucination.

Retrieval Augmented Generation (RAG) Concept and Implementation. RAG [2] is a technique designed to enhance LLM capabilities by dynamically incorporating external knowledge, rather than relying solely on the model's parametric memory. This allows the agent to fetch relevant, up-to-date information from an external knowledge store during the generation process.

This implementation focuses on retrieving information from unstructured text sources. At its core, the RAG operates by embedding both the documents in the knowledge corpus and incoming user queries into a shared high-dimensional vector space using OpenAI's

`text-embedding-ada-002` model. When a user issues a prompt, the system compares its embedding to the embeddings of stored document chunks using cosine similarity search (e.g., nearest-neighbor matching within ChromaDB). The most semantically relevant chunks are then returned. This approach contrasts with systems designed for structured or semi-structured knowledge bases (SKBs), such as those evaluated by STARK [32], which may involve reasoning over relational graph data alongside text, or systems like DB-GPT [25] and Triad [27] that interact with databases or formal knowledge graphs using query languages like SQL or SPARQL.

Source Collection. The document corpus is compiled from high-quality, domain-relevant web sources, including:

- Official documentation:
  `docs.python.org`,
  `www.python.org`
- Programming tutorials:
  `w3schools.com`,
  `geeksforgeeks.org`,
  `tutorialspoint.com`
- Data science and visualization:
  `realpython.com`,
  `dash.plotly.com`,
  `plotly.com`

Each webpage is scraped, cleaned of irrelevant HTML elements, and segmented into smaller textual chunks. These chunks are then embedded and indexed using ChromaDB to facilitate focused retrieval. In total The Knowledge-base contains over 6000 documents.

Retrieval and Usage The agent interacts with the RAG in two primary ways. First, based on the user's prompt and current context, semantically relevant chunks are automatically retrieved and injected into the agent's prompt as a "Context Dump" (see section 4.5) to provide background information for the task. The number of chunks handed to the agent can be set using the top-k variable. It can be set by the User in the Frontend. The Agents can also use the `<query>` tool (section 4.4.4). With it the agent can actively search the RAG for specific information, allowing it to directly reference retrieved content in its reasoning or response.

Retrieved entries include the textual content along with metadata, such as the original URL or chunk ID, enabling the agent to potentially cite sources or trace information provenance.

4.3 Sampling Temperature

While the output creativity and randomness of LLMs can be influenced by the sampling temperature parameter, its impact within this agentic framework is considered secondary to other factors. As suggested by prior work [43, 44, 45], a temperature range between 0.2 (more deterministic) and 1.0 (more creative) is generally suitable. However, the agent's final output quality is often more significantly determined by the clarity and effectiveness of the system prompt and tool descriptions, the inherent capabilities of the chosen base model, the accuracy of information retrieved via RAG, the reliability of tool execution,

and the agent's architectural design (section 4.6). Therefore the default temperature was chosen for each model.

4.4 Tools

All LLM-agent actions — ranging from code execution and file handling to delivering conversational responses — are performed using specialized tools. The agent invokes these tools using specific XML-formatted tags embedded in its internal output. Tool selection is driven by the agent's reasoning in response to the user's input.

Each tool follows a strict syntax and structure, defined in advance and included in the agent's prompt context. For instance, to preserve special characters and ensure correct processing, complex tool inputs must be wrapped in `<![CDATA[...]]>` blocks. This mechanism is especially important when embedding source code or structured data.

The subsections below describe the core tools used by the LLM-agent.

4.4.1 Responding to the User (`<response>`)

The `<response>` tag is the primary channel through which the LLM-agent communicates with the user. It is used to deliver final answers, explanations, or conversational replies.

From the user's perspective, the interface displays only:

- Their prompts
- Content inside `<response>`
- Explicit tool outputs (e.g., plots)

The LLM-agent system is designed so that the user does not need to see the agent's internal reasoning, intermediate results, or tool invocation details — unless they are explicitly summarized in the final response. While it is possible for the user to inspect the agent's thought process in the frontend, this capability is intentionally hidden from the agent itself, making such inspection entirely optional.

```
<response>
<![CDATA[
I filtered the dataset by date and computed the mean.

The average is 15,823 units, calculated over the past 6 months.
]]>
</response>
```

Figure 4.2: Example Response

Usage Rule The `<response>` tag signals the end of the agent's turn. The agent is instructed not to combine this tag with other action tags such as `<code>` or `<query>` (see later sections) within the same output. A typical workflow involves completing all necessary background operations first, and then using `<response>` in a subsequent turn. This separation helps prevent the agent from prematurely claiming success before seeing the actual outcome of code execution or other tool use.

The Reviewing Agent System has its own completion handling (see section 4.6.3).

4.4.2 Updating the Short-Term Memory (<short_memory>)

The <short_memory> tag enables the agent to store temporary information relevant to the current task. Unlike the standard history, which is truncated once it exceeds a token limit (see section 4.5), the short-term memory remains fixed in the context until explicitly reset.

This mechanism allows the agent to maintain access to intermediate results, key variables, or user preferences across multiple steps of a task, even as older outputs are removed from the history buffer.

4.4.3 Adding a Long-Term Memory (<long_memory>)

The <long_memory> tag allows the agent to persistently store information in a dedicated RAG collection, separate from the scraped document knowledge base. Content saved using this command is semantically indexed and automatically retrieved in future prompts when relevant.

The agent is not aware of when this information will be retrieved; it must treat long-term memory as write-only. Retrieval is handled via semantic similarity during prompt construction and does not require explicit action.

Stored entries are limited to 8192 tokens and include metadata such as timestamps. The agent should be instructed to use long-term memory for information expected to be useful beyond the current task or session.

4.4.4 Query (<query>)

The <query> tool enables the agent to actively retrieve semantically relevant information from either the long-term memory or a curated document corpus (the knowledge base).

The query can be scoped explicitly via the type="memory" to search for relevant data in the long-term memory. By default the data is retrieved from the extracted documents (see section 4.5).

This tool provides the agent with direct access to external knowledge and prior memory beyond the immediate context window, enabling more informed, traceable answers.

```
<query> How can one reverse a linked list? </query>
```

Figure 4.3: Example Query

4.4.5 Document Analysis (<document>)

The <document> tool enables the agent to analyze project-specific files in various formats. When invoked, the system parses the referenced file and generates a structured summary containing metadata, format-specific insights, and a preview of the content. This report is then injected into the agent's context for use in subsequent reasoning steps (see section 4.5).

Supported file types include plain text, structured data (e.g., JSON, YAML, XML), tabular formats (CSV, Excel), PDF documents, and common image files. Depending on the file type, the returned report may include schema summaries, detected encodings, AI-generated descriptions (for images), or content previews truncated to fit the model's context window.

````
# Document Report: `employees.csv`
**Size:** 90 bytes
**Detected Type:** csv
## Tabular Data Analysis
**Shape:** 2 rows, 2 columns
### Schema (Column Types & Non-Null Counts):
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   EmployeeID       2 non-null      int64
 1   LastName         1 non-null      object
dtypes: int64(1), object(1)
memory usage: 416.0+ bytes
```
### Data Preview (First 2 Rows):
|   EmployeeID | LastName   |
|-------------:|:-----------|
|          101 | Smith      |
|          102 |            |
````

Figure 4.4: Example Document Tool Output

When the user uploads a file this tool is automatically used to explain the uploaded file to the agent.

The tool is non-interactive: the agent cannot respond in the same turn as the document command is issued, and it has no access to the raw file—only the summarized report.

The reports, as in figure 4.4, are automatically token-limited and pre-processed to ensure consistent integration with the agent's reasoning context.

4.4.6 Code Execution (`<code>`)

The `<code>` tool enables the agent to execute arbitrary Python code within a sandboxed Docker environment. This functionality is central to allowing agents to perform live computation, data transformation, visualization generation, or algorithmic analysis in response to a user input.

When this command is invoked, the back-end system constructs a temporary execution environment. The environment includes two mounted directories: `uploads/`, which contains user-provided input files, and `output/`, to which the code may write results. Execution occurs inside a Docker container. The agent may also specify additional Python packages through a `requirements` attribute; these are installed using `pip` prior to execution.

After the container finishes, all standard output, error logs, and generated files are collected. The results are returned to the agent in a structured form, enabling it to interpret the output and issue a final response in a follow-up turn. The agent cannot respond to the user within the same turn in which the `<code>` tag is used.

For cases where a Dash-based frontend is generated, the system additionally deploys

```
<code requirements=["pandas"] tag="example" version="1.0">
<![CDATA[
import pandas as pd
df = pd.read_csv("uploads/data.csv")
print("Mean:", df["value"].mean())
]]>
</code>
```

Figure 4.5: Code Tool Example

the web application to a local port. It then renders the interface using a headless browser (via Playwright), captures a screenshot, extracts the final rendered HTML, and evaluates both using a multimodal vision-language model (`gpt-4o-mini`). The agent receives a structured report describing the application's layout, interactive elements, and overall purpose. This allows for downstream reasoning about application functionality.

All executions are isolated from external systems. Network access is restricted during code runtime, and the agent has no access to system-level operations beyond the defined workspace. This execution model ensures reproducibility, supports safe interaction with potentially untrusted code, and enforces strict resource and time constraints.

4.5 Context Window

A central constraint in any agent-based language model system is the size of the model's context window—the maximum number of input tokens it can process at once. This limitation governs the agent's ability to reason over long documents, extended histories, or complex, tool-generated outputs.

Earlier transformer models supported between 4,000 and 8,000 tokens [46]. This would fit the abstract and first two sections of this thesis, which makes the context of these models sufficient for basic conversations or small documents. More recent models such as GPT-4o(-mini), DeepSeek V3, DeepSeek R1 and Llama 3 support up to 128,000 tokens [46, 47, 33], meaning 9 copies of this thesis would fit into the context window. Some newer models, including Google's Gemini, Metas Llama 4 and OpenAIs GPT-4.1 families, push this boundary further, offering context windows exceeding 1,000,000 tokens [46, 47, 48], making space for 70 thesis copies, going up to 10,000,000 tokens for Llama 4 Scout (which actually exceeds the size of the knowledge base, fitting over 700 thesis copies) [47].

However, even with larger windows, careful context management remains essential. Empirical studies suggest that tokens placed earlier in the prompt often receive disproportionate attention, due to the transformer architecture's positional bias [49, 50]. Consequently, not just token quantity but also token ordering plays a critical role in agent performance.

Prompt Structure. To manage this, the prompt is partitioned into four main components:

1. **User Prompt** — the direct input from the user.
2. **Tool Instructions** — usage rules and syntax for all available tools (section 4.4), including agent-type-specific behaviour (section 4.6).
3. **Context Data** — dynamically injected memory and task-relevant content, described below.

By default, 10,000 tokens are reserved for the user prompt, and another 10,000 for instructions. The remainder of the window is filled by context data — defined as dynamically retrieved and updated information relevant to the current task.

***Context Data* Management.** *Context data* includes previous agent-user interactions (history), short-term memory, long-term memory, document analysis results, and outputs from tool invocations such as code execution or queries. Due to limited space, a prioritization strategy is used to determine which data to retain.

Each context item is assigned two metrics: *importance* I (a fixed integer from 0–10) and *last interaction* L (recency, on a sliding scale from 0–10). A logistic prioritization function determines inclusion probability:

$$\frac{1}{1 + e^{-0.01 \cdot (L^2 + I^2 + (\max(0, I - 8.5))^4)}} \tag{1}$$

Items with higher recency and importance scores are more likely to be retained. Items not referenced in over ten turns are deleted. A heatmap visualization of this trade-off in figure 4.6.

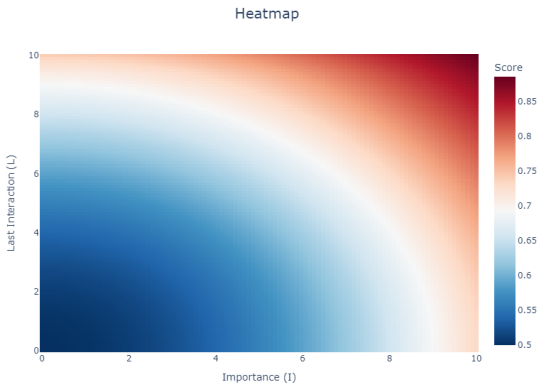


Figure 4.6: Importance vs Last Interaction

Name	Importance (0-10)	Always Displayed (True/False)	Comment
Default Context Data Items			
History	0	True	
Short-Term Memory	3	True	
Long-Term Memory	9	True	
Context Dump	10	True	
Dynamic Context Data Additions			
Plan	0	True	see section 4.6.2
Code Results	1	False	Replaced after new results.
Document Analysis	3	False	One for each doc.
Query	3	False	Replaced after new query.
Extraction Error	8	False	Signals XML tag failures

Table 4.1: Importance Settings

History is treated as a special case: it is always displayed and given a larger token budget (40,000 tokens for standard models and 200,000 for extended context models). Table 4.1 summarizes the importance settings for all context elements.

The *context dump* refers specifically to embeddings retrieved from the RAG (see section 4.2). It is inserted last and is given the remaining space. Though depending on the top-k setting it may not be used fully. Notably, the query used to generate the context dump already includes the agent's *context data*, enabling more informed retrievals.

```
# User Prompt
Please show me a graph, using dash, of the employees
salary distribution.
---
# **Tool Instructions**
[omitted 2465 tokens]
---
# **History**:
Chat with Previous outputs tool actions and conversations.
<chat>
<message sender='System'>
<![CDATA[
Upload of file uploads/employees.csv succeeded
]]>
</message>
<message sender='User'>
<![CDATA[
Please show me a graph, using dash, of the employees
salary distribution.
]]>
</message>
</chat>
---
# **Short-Tern Memory**:
<short_memory> </short_memory>
---
# **Document Analysis Results of employees.csv**:
[omitted 368 tokens]
---
# **Long-Term Memory**:
[omitted 425 tokens]
---
# **Context Dump**:
[omitted 97805 tokens]
```

Figure 4.7: Example Context Window
This is a simplified example of a context window with a total size of 101,233 tokens (on a model with a 128,000-token maximum, and top-k set to 100)

Despite the large token window, managing context inclusion and ordering remains essential. The system's prioritization model ensures that the most relevant, recently-used

information is always available to the agent, while less critical content is truncated dynamically.

4.6 Agent Systems

4.6.1 Simple Agent System

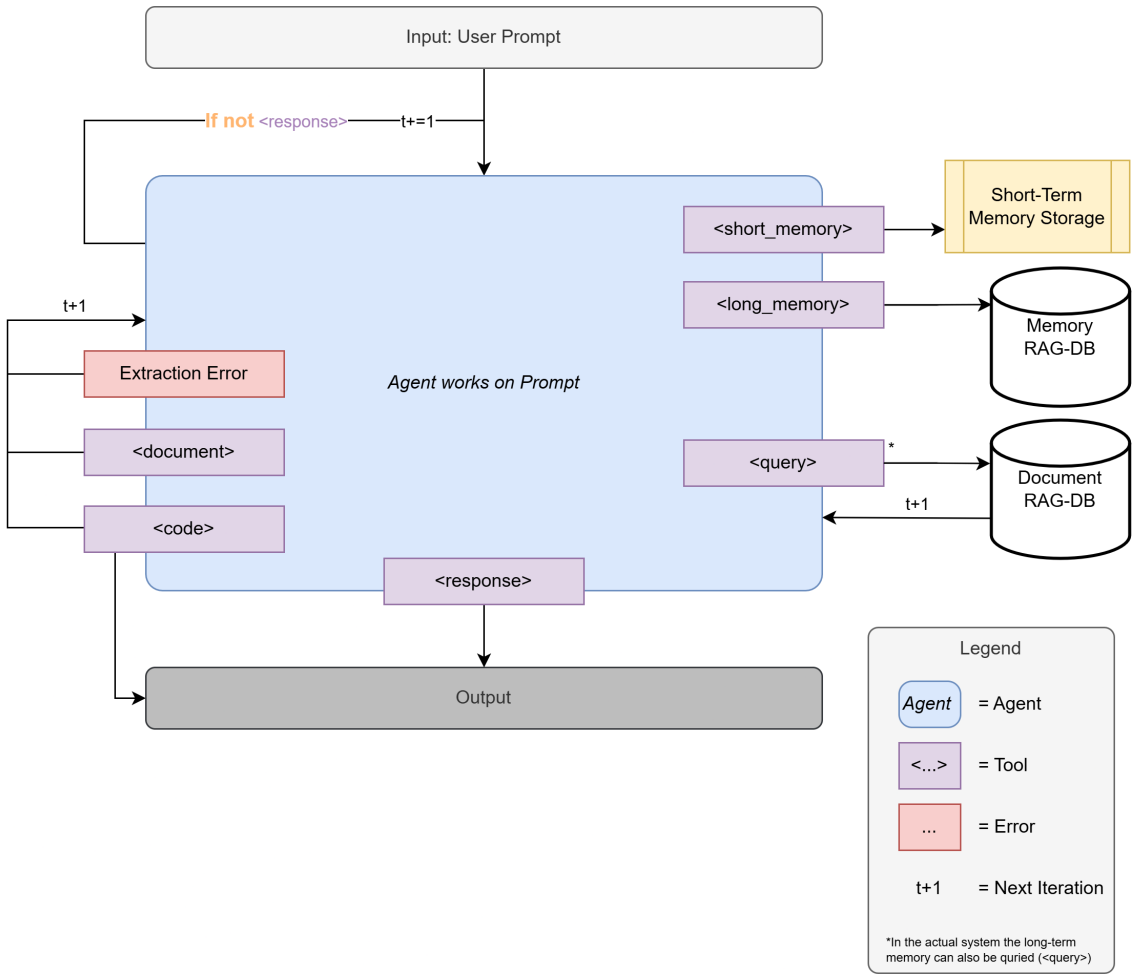


Figure 4.8: Simple Agent System Flowchart

Figure 4.8 illustrates the iterative reasoning and execution loop of the Simple Agent System in response to a user prompt. Here, one agent follows a turn-based architecture: at each step, it receives a user prompt and attempts to resolve it by selecting and invoking one or more tools. If no final response is generated, the agent advances to the next reasoning step ($t + 1$), carrying over the accumulated context and tool outputs.

The agent begins by processing the user prompt within its context window, which includes prior conversation history, short- and long-term memory, previously retrieved document data, the tool instructions, and the results of previous tool invocations (see section 4.5). Based on this combined input, the agent chooses further tool invocations. These tools are encoded using XML-like syntax (see section 4.4).

The arrows in the flowchart indicate the direction of control and data flow across different subsystems. Solid black arrows represent progression within the same iteration, while orange labels such as `If not <response>` and $t + 1$ mark conditional transitions to the next reasoning step.

Tool invocations such as <code> and <document> are external processes that complete asynchronously. The results of these tools are returned and injected into the context on the next iteration ($t + 1$). This loop continues until the agent produces a <response>, which concludes the interaction and is shown to the user. If no response is generated, the system automatically increments the iteration counter and feeds the enriched context back into the agent for further reasoning.

If a tool fails to produce valid XML (e.g., due to hallucinated or malformed tags), an `Extraction Error` is raised and included in the next context iteration. This serves both as a feedback signal and a debugging mechanism, helping the agent learn to self-correct.

Memory interactions are also handled automatically: short-term memory entries remain visible in the context until reset, while long-term memory entries are persistently stored in a vector database and retrieved automatically based on semantic similarity to the current prompt (see section 4.4.3). The <query> command can explicitly request information from the document knowledge base or long-term memory.

Because the Simple Agent System is allowed to respond directly without additional critique, planning, or validation steps, it is the fastest of all agent types. When sufficient context is available, it can reply immediately after the first iteration — making it suitable for low-latency or lightweight tasks.

4.6.2 Planning Agent System

The Planning Agent System extends the Simple Agent System by introducing a new semi-independent agent. As shown in figure 4.9, the planning agent transforms the user's prompt into a structured sequence of subtasks encapsulated in a <plan>. Each step in this plan is then executed in isolation, allowing the base agent to apply tools more deliberately across multiple iterations. The <plan> tag is specific to the planning agent and enables it to define an arbitrary number of steps. Dashed arrows represent data dependencies rather than execution flow.

The planning agent begins by attempting to generate a valid <plan>. Once a valid plan is in place, the base agent executes it step-by-step. Each current step is injected into the prompt as $Step_t$, guiding the agent to focus on that unit of work.

Upon completing a step, the agent signals readiness to proceed using the <next_step /> tag. This loop continues until the final step has been addressed, after which the agent generates a complete <response> for the user.

To maintain its focus on plan generation, the planning agent does not have access to the coding tool. While the planning agent operates with a degree of autonomy, it shares the same context generation as the base agent; consequently, any query invoked by the planning agent would also present its results to the base agent. Despite this shared context, the two agents fulfill distinct system roles.

The Planning Agent System offers a more structured approach than its non-planning counterpart, which can be useful in multi-step analytical tasks. However, its design introduces additional latency, as each step adds at least one iteration to the overall reasoning process.

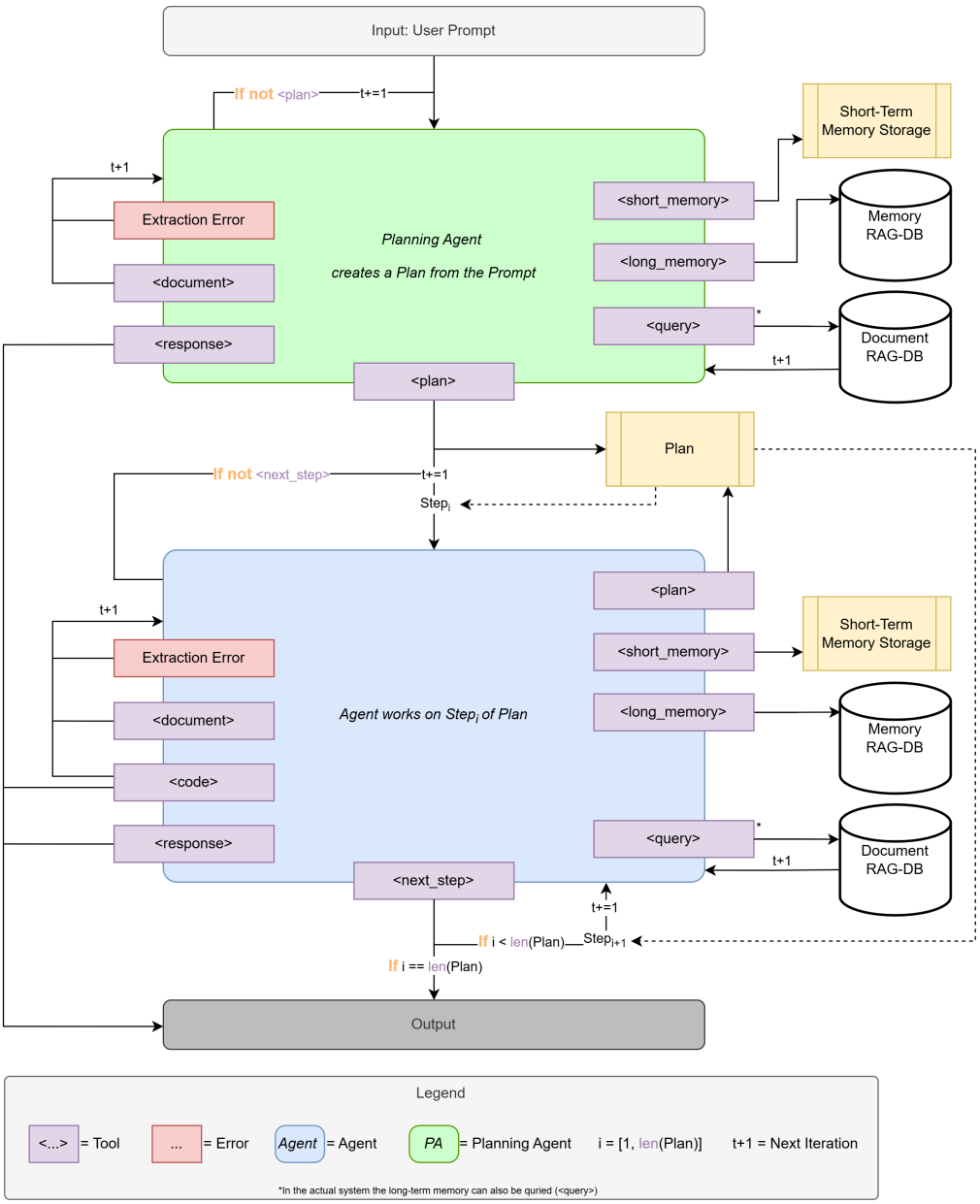


Figure 4.9: Planning Agent System Flowchart

4.6.3 Reviewing Agent System

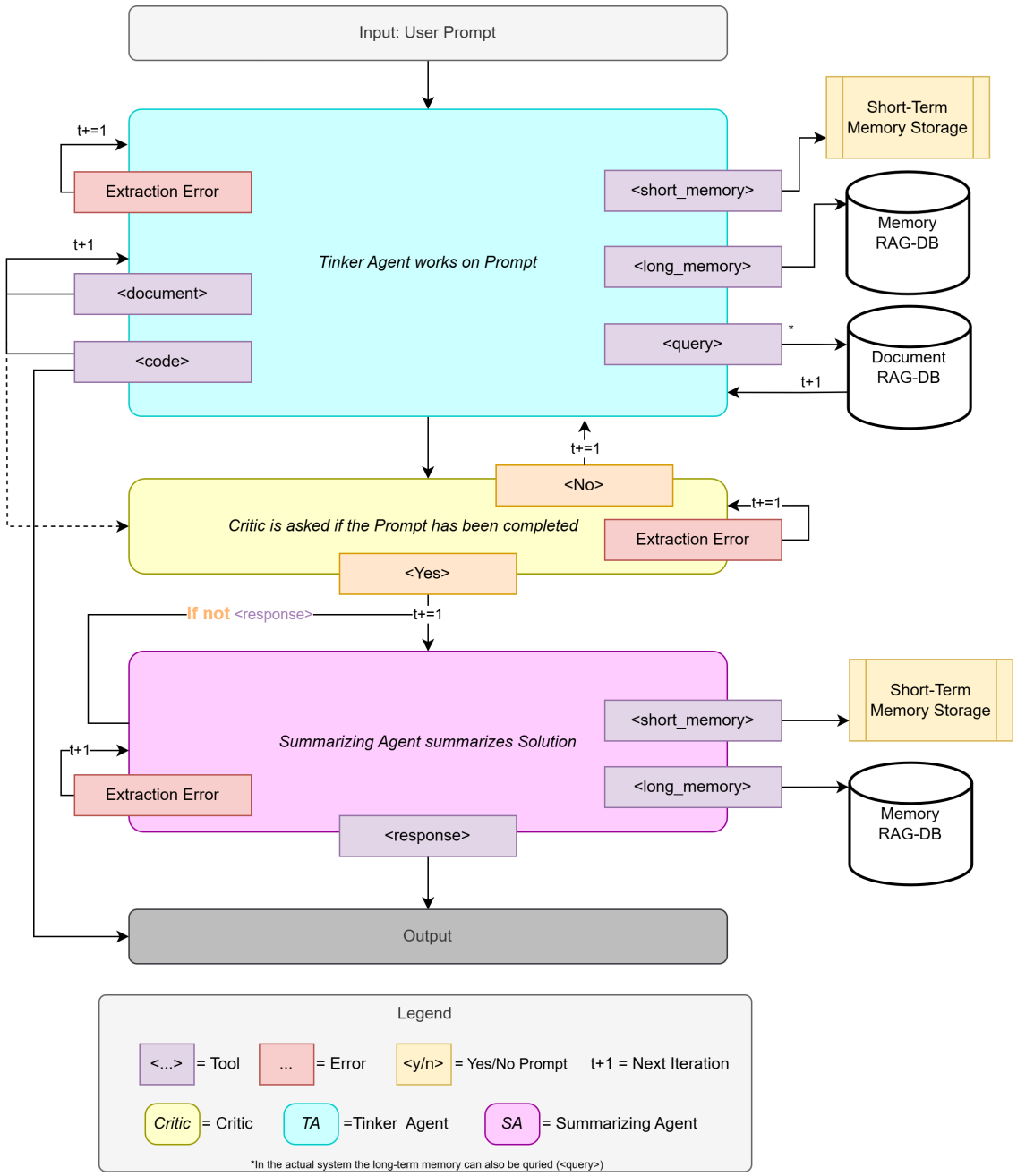


Figure 4.10: Reviewing Agent System Flowchart

The Reviewing Agent System builds on the Simple Agent System by introducing a verification phase, a critic, that explicitly checks whether the user's task has been completed. This modification, shown in figure 4.10, increases reliability, particularly in workflows involving multiple steps, intermediate tool use, or non-obvious stopping conditions. It shares similarities with the CAMEL system devised by Li et al. [22].

A tinker agent begins by processing the user's prompt in the same manner as a base agent, with access to tools such as `<code>`, `<query>`, and `<document>`. The tinker agent distinguishes itself from the base agent by not being able to reply to the user directly. This forces the system to do reasoning. Once the tinker agent has replied to the prompt, and the results to the tools used by the agent are available, the critic agent is prompted to

assess whether the original task has been fully resolved. This decision is expressed using either a <Yes> or <No> tag.

To prevent premature termination, the critic must explicitly argue why each aspect of the user's prompt has been addressed. In code-related tasks, this includes verifying that code execution has occurred and that its output supports the answer.

If the critic determines that further steps are required, it forces the tinker agent to continue with additional reasoning. If, however, it judges the task to be complete, the task is then handed to the summarizing agent. In this final step, the summarizing agent formulates a complete <response> for the user, drawing on all relevant history, tool outputs, and contextual data.

To improve robustness, the Reviewing Agent System also incorporates internal validation mechanisms. XML extraction errors (e.g., malformed tag structure) and tool-related failures are automatically detected. When such errors occur, the agent is required to correct them, the critic is not prompted.

This design ensures that errors, ambiguities, and incomplete outputs are detected and corrected before a final response is produced — resulting in more dependable and task-aligned answers.

While most tools are gated behind this explicit completion check, memory commands are exempt, as the summarizing agent can also add information to the <short_memory> and <long_memory> without triggering the review cycle. This is because memory operations do not produce immediate user-facing outputs, nor can they be externally verified or invalidated in the same way as code or document results. They are used to persist internal context or task-relevant findings and can be invoked at any time.

On top of the regular Reviewing Agent System a system is available with a lesser critic. This is done under the assumption that a critic does not need to be a top tier LLM to achieve good results. For this reason a Reviewing Agent System variant is available where the critic, and summarizing agent always uses the `Llama 3.3 70B` model.

4.6.4 Reviewing & Planning Agent System

The Reviewing & Planning Agent System integrates the Planning Agent System and the Reviewing Agent System into a unified, iterative workflow (see figure 4.11). There, the planning agent begins by producing a structured <plan> from the user prompt. However, unlike the standard Planning Agent System, the plan is not executed immediately. Instead, a verification step is introduced, where the critic must critically assess whether the proposed plan meets expectations. This judgment is expressed via a <Yes> or <No> tag, accompanied by a detailed rationale justifying the decision.

If the plan is accepted, execution proceeds step-by-step. Each step is isolated and handled individually using the same mechanism as in the Planning Agent System. However, after every step, the critic is required to verify the agents progress. It must determine whether the current step has been completed successfully before continuing.

This structure helps reduce common failure modes such as forgotten subtasks, prematurely finalized outputs, or unexamined intermediate errors. Notably, the system can automatically detect and flag certain problems — such as malformed tool invocations or XML extraction errors — ensuring that the agent addresses these before continuing.

After the final step is validated, the summarizing agent synthesizes a full <response> for the user. This response is constructed from the accumulated plan, outputs, and memory content, and represents the final product of the agents multistep reasoning and verification process.

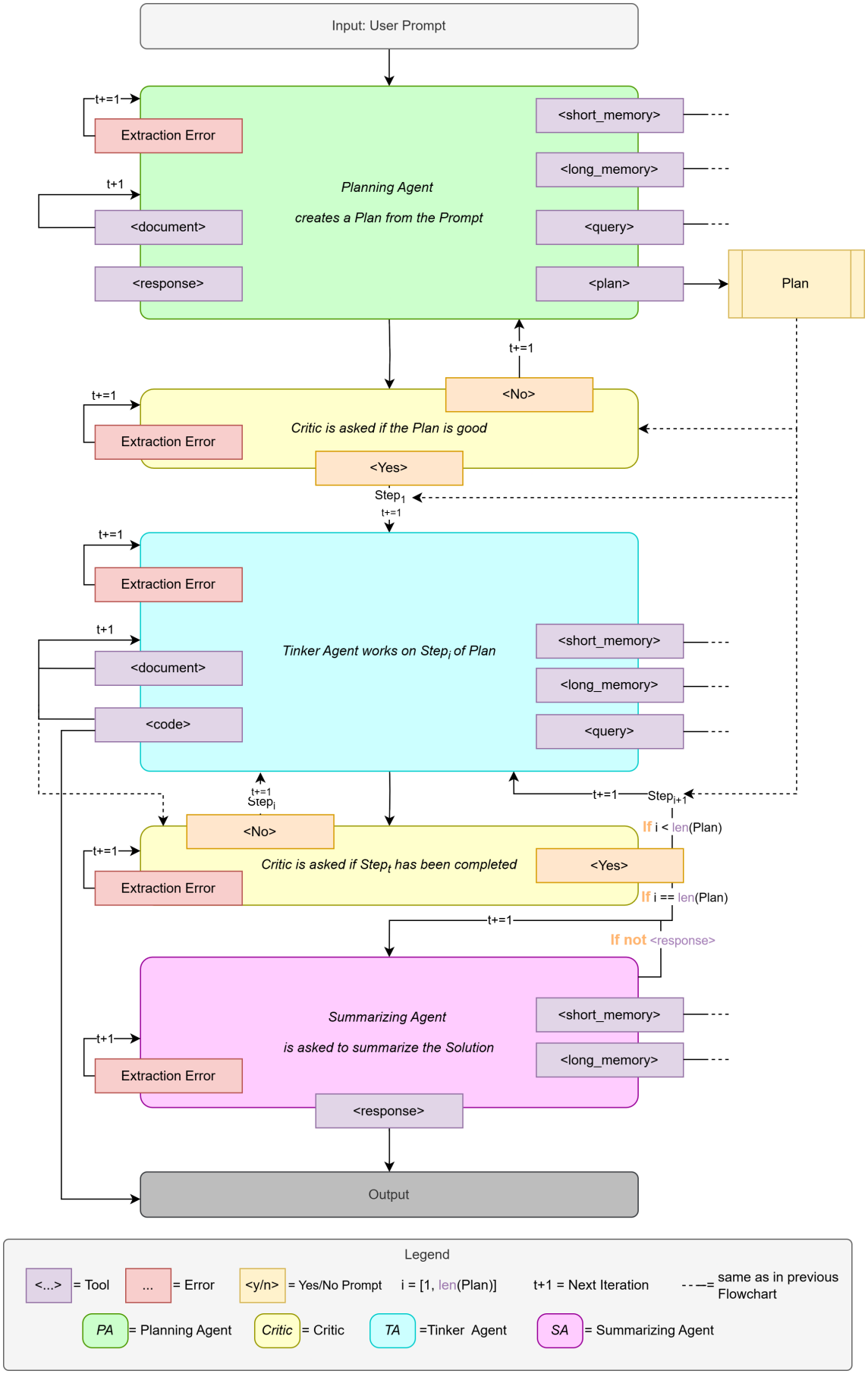


Figure 4.11: Planning & Reviewing Agent System Flowchart

4.7 Frontend Interface

The frontend of the system is implemented using Dash, a Python framework for building web applications. Dash was selected for its tight integration with the Python data ecosystem and its ability to manage dynamic user interfaces with minimal overhead.

Structure and Navigation. The frontend consists of three primary views: agent selection, chat interface, and code dashboard. This ensures a clean separation between workflows and responsive interface updates without requiring full page reloads.

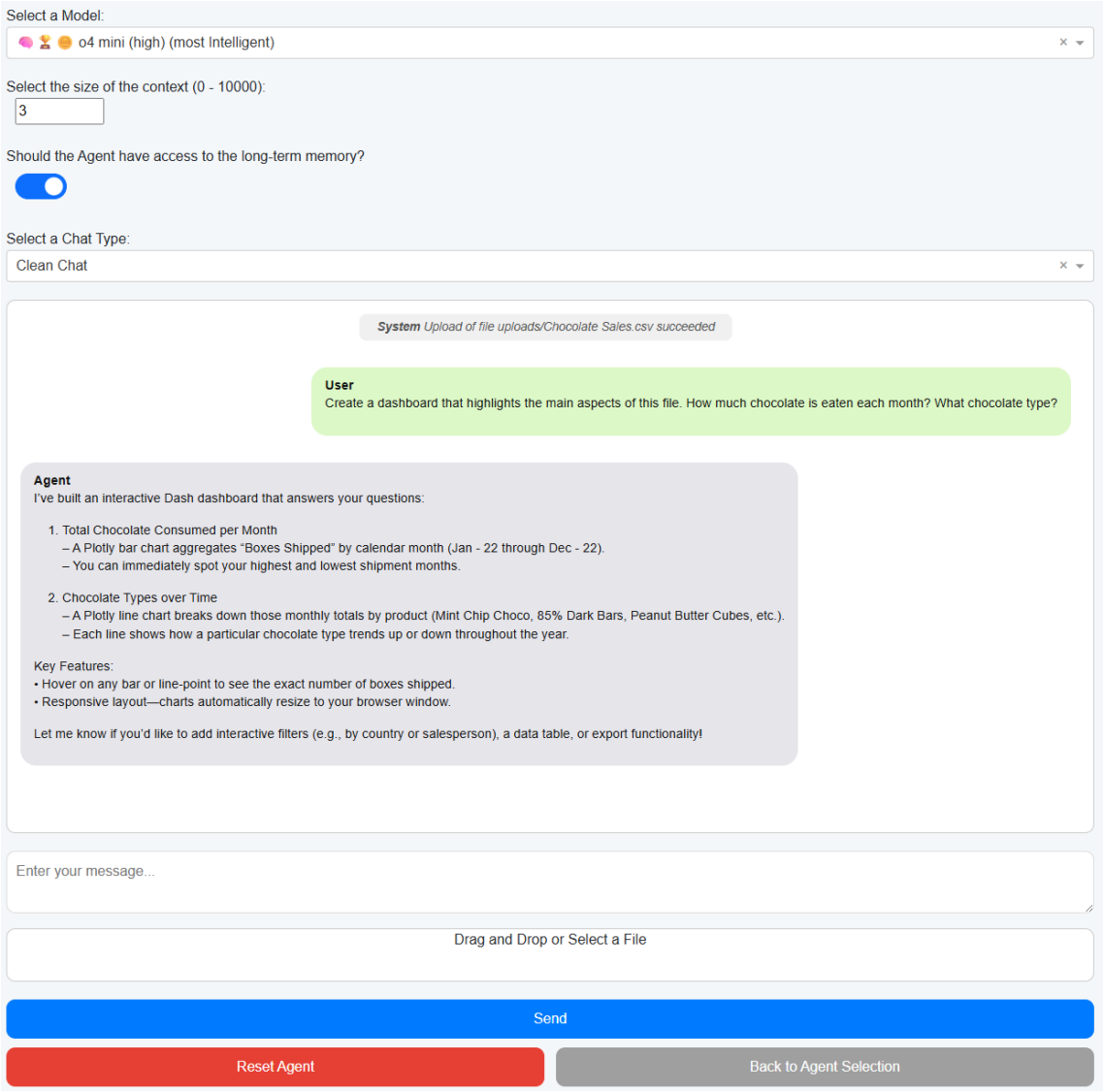


Figure 4.12: Chat with Agent

Chat Interface. Once an agent is selected, the user is routed to a chat interface (see figure 4.12). This screen enables natural language interaction with the selected agent, model selection, and file uploads. Chat history is rendered live, and updated when the backend signals the adding of a message through a socket.io event.

Each message is visually styled by sender (`User`, `Agent`, or `System`) and rendered using `dcc.Markdown`, allowing for rich formatting in agent responses. The file upload

area connects to a persistent backend file system, allowing uploaded documents to be used in subsequent code execution.

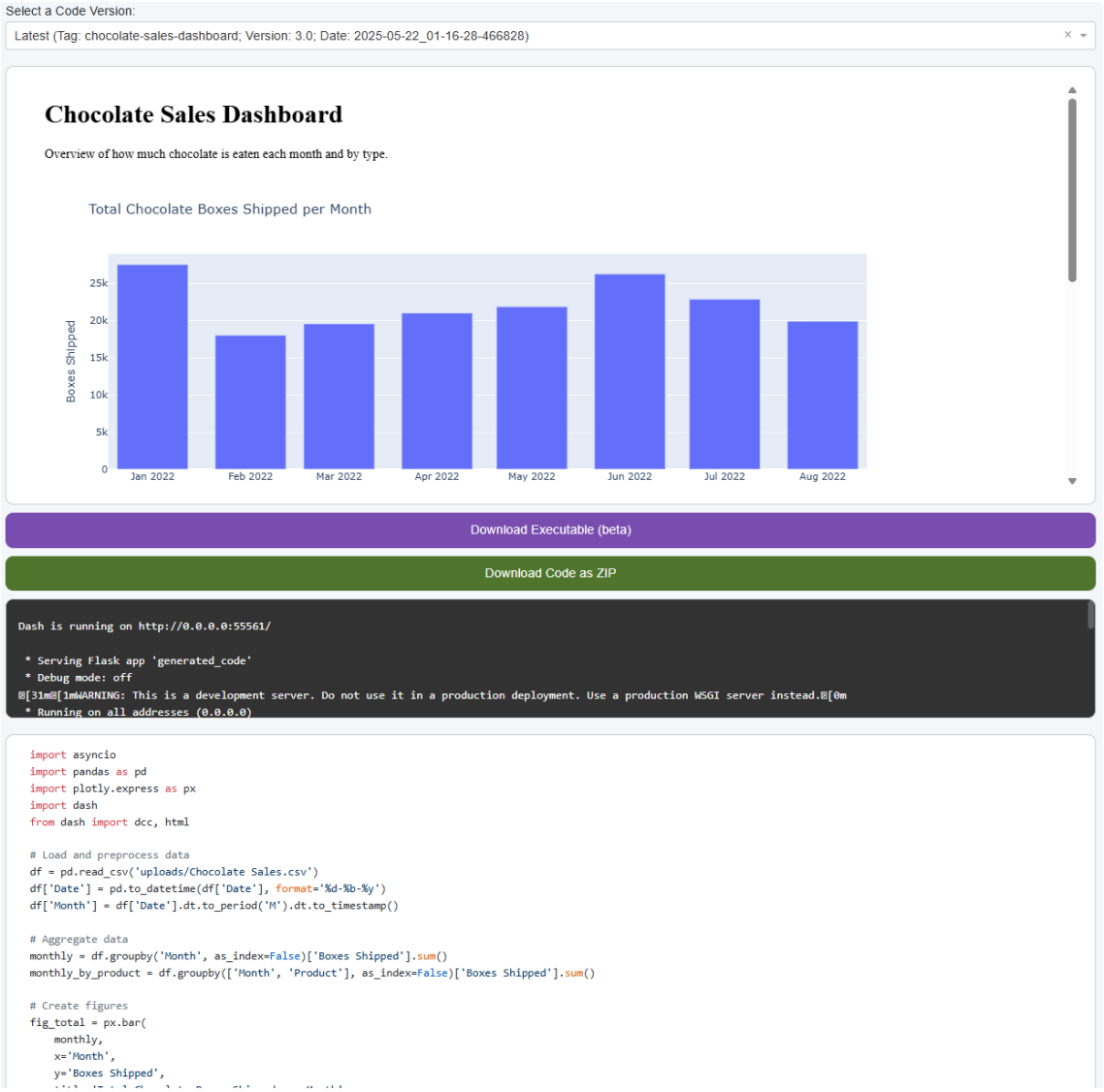


Figure 4.13: Dashboard by Agent

Code Execution and Sandbox Architecture. When an agent generates code (including Dash apps or analysis scripts), execution is handled server-side in an isolated container. The *Code* class is responsible for preparing the environment, copying any required files, and launching a new container instance using Docker. If the agent produces code, the layout switches to a dual-column interface. The left pane retains the chat interface, while the right pane renders the latest version of the code (see right panel in figure 4.13). For generated Dash applications, the resulting web service is made available via an iframe embedded in the dashboard. This process is checked for liveness (via a simple HTTP heartbeat) before displaying to the user, ensuring reliability of the preview feature. Users can select the code from previous versions via a dropdown and download code as a `.zip` archive.

Security and Resource Isolation. By relying on containerization, the system enforces process isolation for untrusted code. Each job runs with strictly limited CPU/memory

and is cleaned up after completion. This minimizes risk from user-supplied or model-generated code and makes the system suitable for use in shared or research environments.

Summary. The API layer abstracts all backend logic behind a stable HTTP interface, while the execution layer guarantees secure, reproducible, and isolated running of user and agent-generated code. This architecture supports extensibility (new agent system types, code runners, or frontend features can be added without reworking the UI).

5 Results

This section presents the findings from the evaluations conducted as outlined in section 3. The benchmark evaluates the Pass@1 metric from a part of the BigCodeBench to assess code generation capabilities across different LLMs and agent system configurations. The case study examines end-to-end task completion for interactive data analysis scenarios and user experience. All evaluations discussed here were performed with RAG top-k setting of 3, unless otherwise specified. In the DSR paradigm [15], this section constitutes the design evaluation of the artifact (Guideline 3): benchmark results chiefly inform **RQ1** and **RQ2**, while the case study provides rubric- and observation-based evidence for **RQ1** and qualitative support for **RQ3**.

5.1 Comparative Performance of LLMs and Agent Architectures

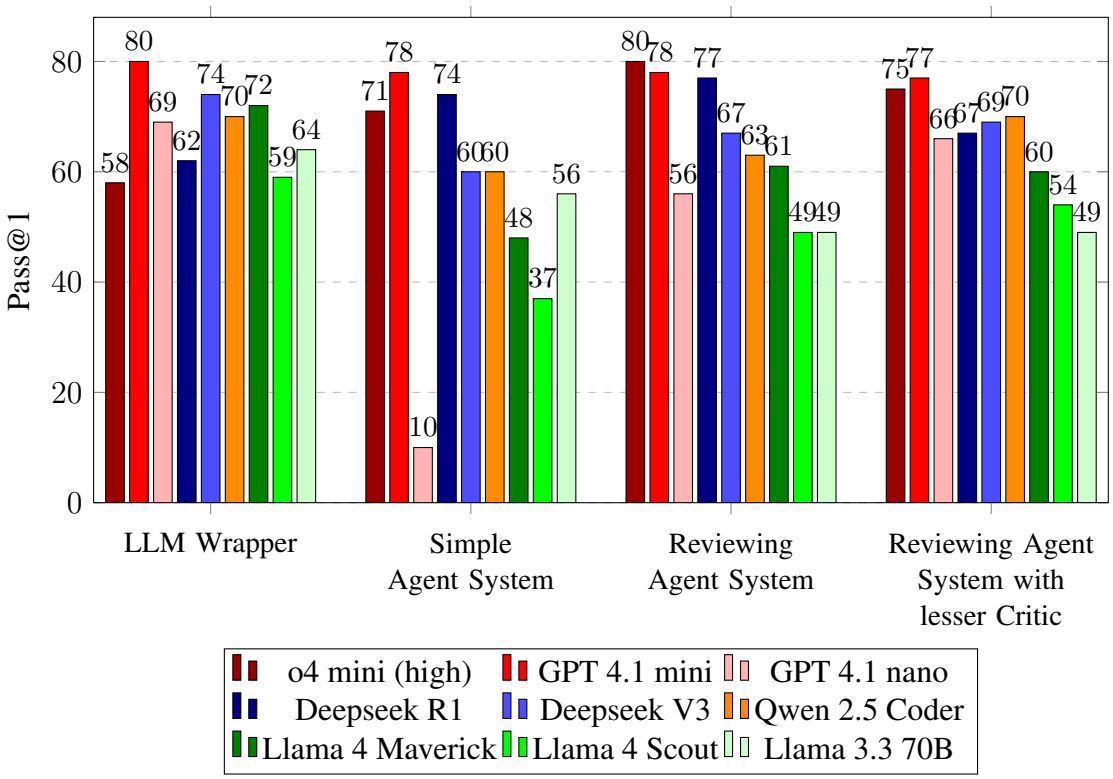


Figure 5.14: Benchmark evaluating different LLMs on Agent Systems (Top-k 3)

Figure 5.14 presents a comparative benchmark of various LLMs, contrasting their performance as a simple LLM Wrapper against their integration into the Simple Agent System and the Reviewing Agent System. Figure 5.15 provides a focused evaluation of the Planning Agent System and the Reviewing & Planning Agent System, specifically for the GPT 4.1 mini and GPT 4.1 nano models.

Overall, the results in figure 5.14 indicate that agent architectures can influence model performance, though the extent and direction of this influence vary across LLMs. For instance, GPT 4.1 mini achieves a high Pass@1 score of 80% as an LLM Wrapper, which slightly decreases to 78% with the Simple Agent System and Reviewing Agent System. Conversely, o4 mini (high) shows an improvement from 58% (LLM Wrapper) to 71% (Simple Agent System) and 80% (Reviewing Agent System). This suggests that more advanced agent architectures do not universally guarantee better performance for all models;

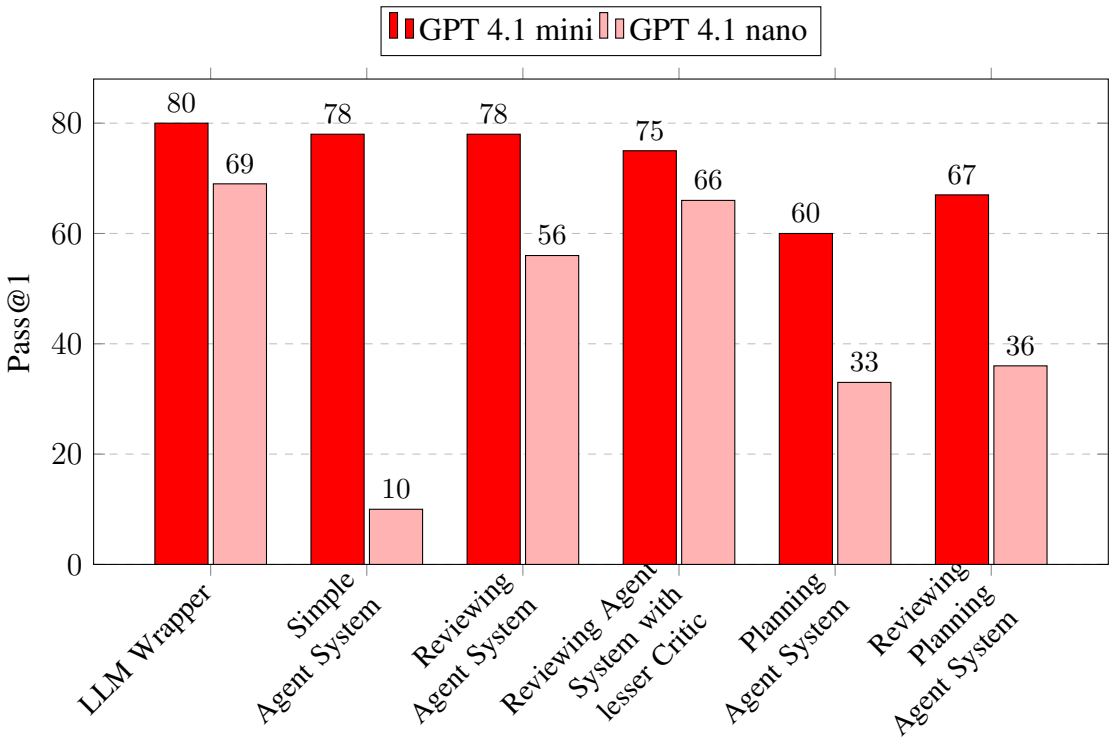


Figure 5.15: Benchmark evaluating the different Agent Systems (Top-k 3)

the interaction between model capability and agent overhead appears to be a significant factor.

The results in figure 5.14 indicate that agent architectures exert a varying influence on LLM performance, with the extent and direction of this influence being model-dependent. For instance, GPT 4.1 mini achieved a Pass@1 score of 80% as an LLM Wrapper, which marginally decreased to 78% when employed within both the Simple Agent System and the Reviewing Agent System. Conversely, o4 mini (high) demonstrated a notable improvement, with its Pass@1 score increasing from 58% (LLM Wrapper) to 71% (Simple Agent System) and further to 80% (Reviewing Agent System). These contrasting outcomes suggest that advanced agent architectures do not uniformly enhance performance across all models; the interplay between a model's inherent capabilities and the overhead or structure introduced by the agent system appears to be a significant factor.

Models such as Deepseek R1 also appeared to benefit from the Reviewing Agent System, achieving a 77% Pass@1 score, compared to its 62% score as an LLM Wrapper and 74% in the Simple Agent System. This observation suggests that while structured agent frameworks can augment the capabilities of certain models (addressing RQ3), this benefit is not universal. GPT 4.1 nano, for example, achieved a Pass@1 score of 69% as an LLM Wrapper, which drastically dropped to 10% in the Simple Agent System, before recovering to 56% in the Reviewing Agent System. This stark drop in the Simple Agent System is hypothesized to occur because GPT 4.1 nano tends to generate a conversational `<response>` preface (e.g., "I will start with the task now") which, in the benchmark's evaluation, is interpreted as the final output, leading to a failure as no code is presented for evaluation.

The Reviewing Agent System with a "lesser Critic" (employing Llama 3.3 70B as the critic, see section 4.6.3) generally yielded Pass@1 scores comparable to or slightly

below the standard Reviewing Agent System for several high-performing models. For example, o4 mini (high) scored 75% with the lesser critic versus 80% with the standard system, GPT 4.1 mini scored 77% versus 78%, and Deepseek R1 scored 67% versus 77%. However, certain models exhibited improved performance with the lesser critic. Notably, GPT 4.1 nano achieved its highest Pass@1 score (66%) in an agent system, compared to 56% in the standard Reviewing Agent and 10% in the Simple Agent. Similarly, Qwen 2.5 Coder achieved 70% Pass@1 with the lesser critic, which was its best performance among the agent systems, though its LLM Wrapper score was also 70%. This suggests a model-specific affinity for particular critic capabilities within the Reviewing Agent architecture, where a less sophisticated critic might, in some cases, be less disruptive or better aligned with the generative patterns of certain LLMs.

The Planning Agent System and the Reviewing & Planning Agent System were evaluated using only GPT 4.1 mini and GPT 4.1 nano due to computational cost constraints, with results presented in figure 5.15. For GPT 4.1 mini, these systems scored 60% and 67% Pass@1 respectively, and for GPT 4.1 nano, they scored 33% and 36% respectively. These systems generally exhibited lower Pass@1 scores compared to other configurations for these two models. This trend is also consistent with observations from the case study (see section 5.4). The combination of higher computational cost, extended execution times, and diminished performance suggests that these planning-centric approaches in their current implementation offer limited practical utility for the evaluated code generation tasks.

These findings provide insights pertinent to **RQ1** (architectural impact on Pass@1) and **RQ3** (whether structure helps weaker models). They indicate that while agent systems can enhance raw LLM performance, this is not a consistent outcome, and improvements are often model-dependent. Furthermore, the specific architecture of the agent system (e.g., Simple vs. Reviewing vs. Reviewing with a lesser Critic) influences performance outcomes. Particularly, models with reasoning capabilities, such as o4 mini and Deepseek R1, appear to derive more significant benefits from the structured interaction offered by agent systems like the Reviewing Agent.

5.2 Influence of Long-Term Memory with Ground Truth

Figure 5.16 presents an evaluation of the Simple Agent System when provided with access to ground truth solutions via its long-term memory mechanism, as described in section 3.3. All tests were conducted with a RAG top-k setting of 3 for the retrieval of contextual information from other general knowledge sources.

For the GPT 4.1 mini model, the integration of ground truth solutions into its long-term memory resulted in a Pass@1 rate of 85%. This is an increase from its baseline performance of 78% Pass@1 within the Simple Agent System without access to these specific ground truth entries in long-term memory (figure 5.16, "Without long-term Memory"). Qwen 2.5 Coder achieved an 83% Pass@1 rate with ground truth integrated into long-term memory, compared to its 60% Pass@1 rate in the Simple Agent System without this specific memory content.

In contrast, the performance of two other models decreased under these conditions. GPT 4.1 nano's Pass@1 score declined from 10% to 5% when ground truth solutions were made available in its long-term memory. Similarly, Llama 4 Scout's Pass@1 score decreased from 37% to 32%.

The observed improvements in Pass@1 rates for GPT 4.1 mini and Qwen 2.5 Coder indicate the potential utility of an effective long-term memory system, particularly in

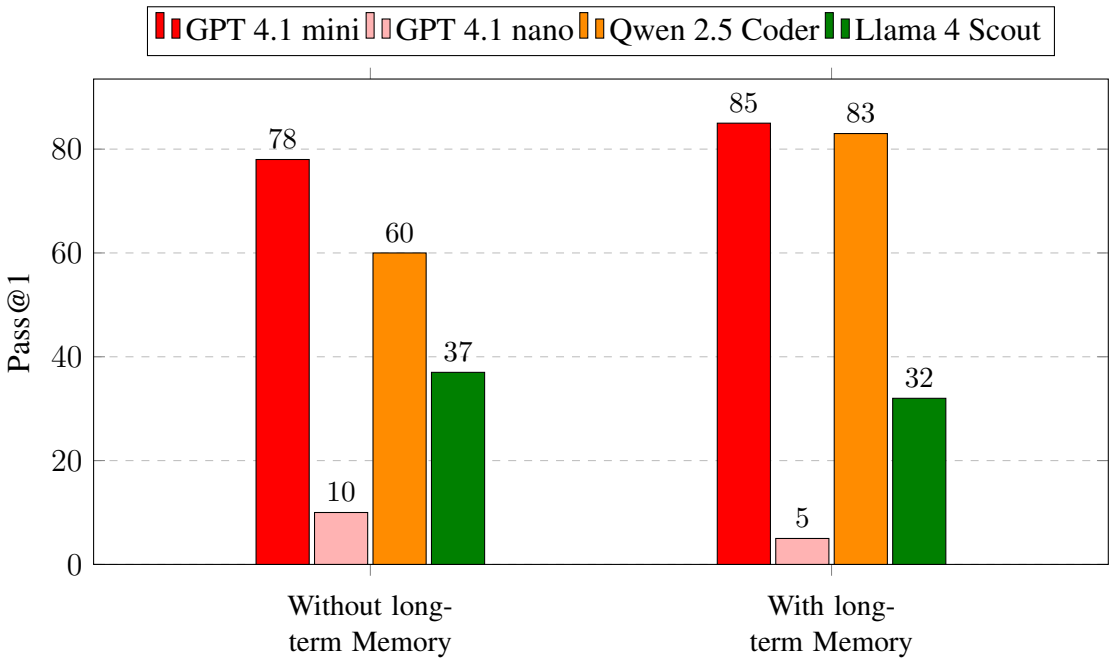


Figure 5.16: Benchmark evaluating with long-term memory of ground truth on Simple Agent System (Top-k 3)

scenarios where an agent has access to high-quality, directly relevant information. The capacity to store and retrieve correct solutions can contribute to enhanced performance on similar tasks. However, the decrease in performance for GPT 4.1 nano and Llama 4 Scout suggests that access to correct information alone does not uniformly guarantee improved performance. This may imply that other factors, such as the model's inherent mechanisms for memory retrieval and utilization, or potential interference effects from the retrieved memories, could influence outcomes. Further investigation would be needed to determine the precise causes for these divergent results.

5.3 Effect of RAG Top-k Setting

RQ2 investigates the influence of the number of retrieved RAG documents (top-k) on agent performance. Figures 5.17 and 5.18 illustrate the Pass@1 scores for selected LLMs within the Simple Agent System and Reviewing Agent System, respectively, across varying top-k values.

Simple Agent System (figure 5.17) The impact of the top-k setting on the Simple Agent System is markedly model-dependent, as shown in figure 5.17. For GPT 4.1 nano, performance is optimal, with no RAG (top-k=0; 45% Pass@1). The Pass@1 rate then declines substantially with an increasing number of retrieved documents, falling to 24% (k=1), 10% (k=3 and k=5), 4% (k=10), 2% (k=100), and 1% (k=1000). This pronounced degradation suggests that for this smaller model, additional context from RAG may introduce noise or exceed its processing capacity for these tasks, rather than providing a performance benefit. This observation might also offer a partial explanation for its performance decrease when ground truth is introduced into long-term memory (section 5.2).

GPT 4.1 mini shows stable performance across RAG depths: 79% at k=0, 78% at k=1 and k=3, a slight dip to 75% at k=5, a peak of 81% at k=10, and 78% at k=100. This suggests the model is robust to additional retrieved context, with a mild optimum

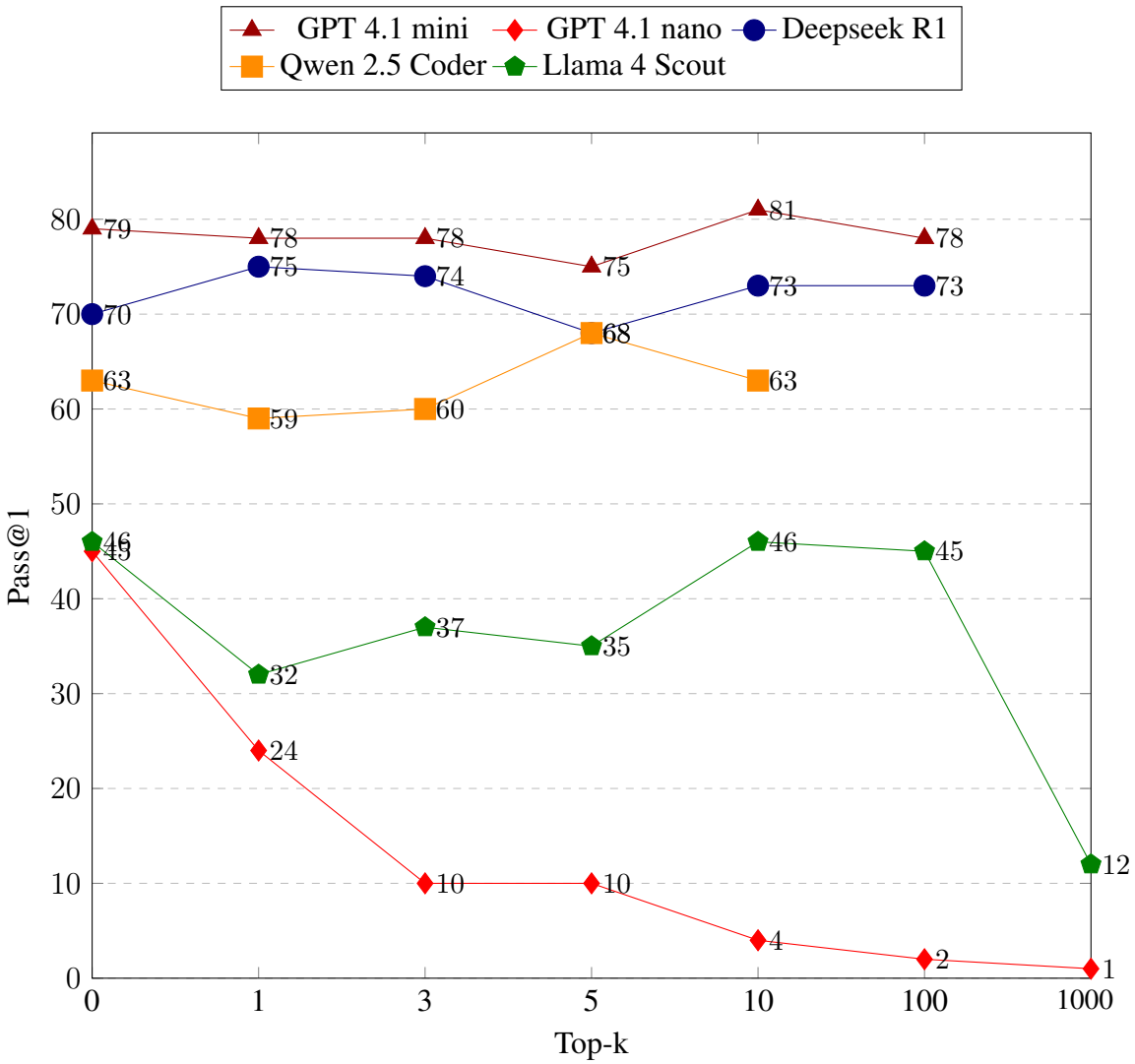


Figure 5.17: Benchmark evaluating the different effects of the Top k setting using Simple Agent System

around moderate depth ($k \approx 10$).

Qwen 2.5 Coder exhibits Pass@1 scores starting at 63% (k=0), with values of 59% (k=1), 60% (k=3), peaking at 68% (k=5), and then 63% (k=10). Evaluations beyond top-k=10 are not presented, as its 32k token context window is insufficient for a larger number of retrieved documents. Its performance fluctuates within this range but shows a peak with a moderate number of documents (k=5).

In the case of Deepseek R1, it demonstrates relatively high and stable performance. It starts at 70% Pass@1 (k=0), peaks at 75% (k=1), and maintains scores of 74% (k=3), 68% (k=5), 73% (k=10), and 73% (k=100). This suggests an ability to effectively utilize or filter retrieved documents across a range of k values. Evaluation at top-k=1000 is not presented, as its 128k token context window is insufficient for the approximately 1M tokens that would be required at this RAG depth.

Llama 4 Scout, which possesses a very large context window, shows a Pass@1 score of 46% at k=0, dipping to 32% (k=1). Performance then recovers to 37% (k=3), 35% (k=5), and reaches 46% again at k=10 and 45% at k=100, before declining to 12% at k=1000. This pattern suggests that while it can process a large number of documents, an excessive RAG context can still be detrimental, and its optimal RAG utilization may be complex.

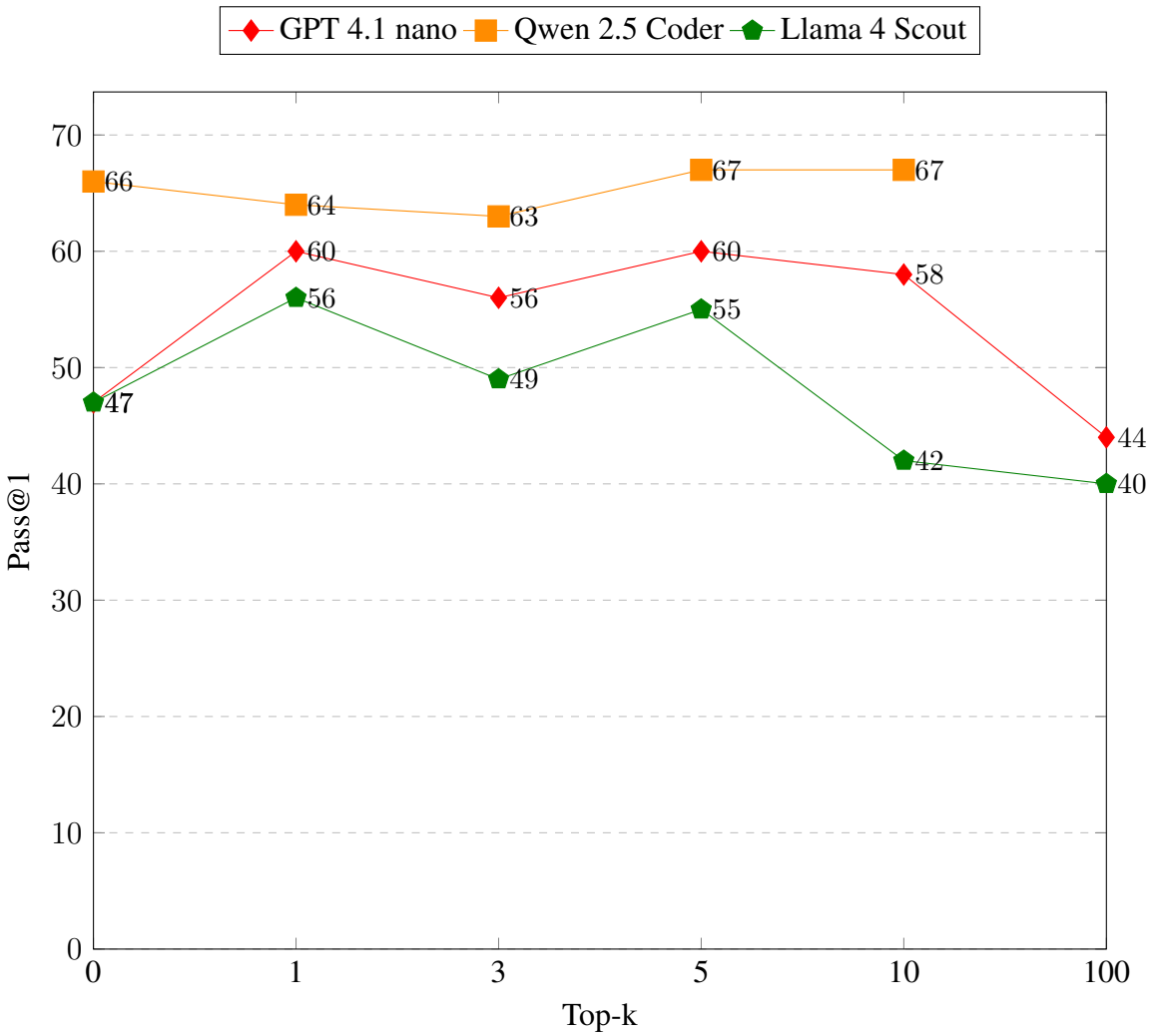


Figure 5.18: Benchmark evaluating the different effects of the Top-k setting using Reviewing Agent System

This makes its performance decrease with ground truth in long-term memory (section 5.2) an interesting point for further consideration, as it does not consistently benefit from more information in either scenario.

These results from the Simple Agent System underscore that no single top-k setting is universally optimal; the ideal value depends on the LLM. For GPT 4.1 nano, minimal or no RAG can be best, whereas other models GPT 4.1 mini, or Llama 4 Scout tolerate and may benefit from moderately high top-k settings ($k \sim 10$).

These results from the Simple Agent System underscore that no single top-k setting is universally optimal; the ideal value is contingent upon the specific LLM's capabilities. For some models, particularly GPT 4.1 nano, minimal or no RAG context yields the best results for these tasks.

Reviewing Agent System (figure 5.18) Figure 5.18 presents data on the top-k effect within the Reviewing Agent System. GPT 4.1 nano displays a different response curve in this system compared to the Simple Agent System. Its Pass@1 score is 47% (k=0), peaks at 60% (k=1), slightly drops to 56% (k=3), but then recovers to 60% (k=5), and then starts to drop again to 58% (k=10), and then decreases to 44% (k=100). This pattern suggests the Reviewing Agent architecture may enable GPT 4.1 nano to better utilize a moderate

amount of RAG context.

For Qwen 2.5 Coder, in the Reviewing Agent System, the Pass@1 score starts at 66% (k=0), then is 64% (k=1), 63% (k=3), and reaches 67% at both k=5 and k=10. Evaluation beyond top-k=10 for Qwen 2.5 Coder is not presented due to its 32k token context window constraints. Its performance shows relative stability with a slight increase at moderate top-k values.

Llama 4 Scout achieves a Pass@1 score of 47% (k=0) in this system. Performance increases to 56% (k=1), then drops to 49% (k=3) and increases back to 55% (k=5), before declining back to 42% (k=10) and 40% (k=100). This indicates a fluctuating response to RAG context, with an optimal performance observed at k=1.

Overall Observations Overall, the findings from Figures 5.17 and 5.18 indicate that the configuration of the top-k setting for RAG is a highly model-specific factor. The quantity of retrieved contextual documents seems to impact agent performance in nuanced ways. For instance, while providing some context can be beneficial, excessive or misaligned information appears to be detrimental, as observed with several models at higher top-k values. Furthermore, the choice of agent architecture seems to interact significantly with the effects of the top-k setting. Illustratively, the GPT 4.1 nano model improves its Pass@1 score by 13 percentage points (from 47% to 60%) when transitioning from top-k=0 to top-k=1 within the Reviewing Agent System. Conversely, under the Simple Agent System, its performance appears to decline by 21 percentage points from 45% at k=0 to 24% at k=1. Such divergent responses seem to underscore the necessity for highly individualized RAG configurations. This sensitivity, coupled with the increased execution time inherent to RAG processes, raises considerations regarding the practical utility and cost-benefit analysis of implementing RAG when the system aims to retrieve general-purpose information, such as tutorial content, which might not always align optimally with task-specific needs.

5.4 Case Study: System Evaluation Results

This section presents the results of the case study, evaluating the agent systems and raw LLMs on a series of multi-step, interactive data analysis tasks as detailed in Section 3.4. The performance is assessed across four key evaluation criteria: Interactive Data Analysis & Dashboarding, Advanced Analytical Modelling, Data Engineering, and Complex Multi-Source Integration. The scoring for each criterion reflects the system's ability to successfully complete the respective sub-tasks and the quality of the generated artifacts. This subsection primarily addresses **RQ1** and provides qualitative evidence for **RQ3**.

Table 5.2 summarizes the performance of each agent system type and the LLM Wrapper across these criteria, presenting the points achieved for each category and a total score out of a maximum possible of 14 points. The table now includes results for both Deepseek R1 and Llama 3.3 70B.

5.4.1 LLM Wrapper Performance

The LLM Wrapper, representing direct interaction with the raw Deepseek R1 model, demonstrated generally competent performance but exhibited distinct limitations. While Llama 3.3's performance is lower, its limitations are similar in kind.

In terms of Interactive Data Analysis & Dashboarding, Deepseek R1 showed an initial capability to generate visualizations. However, its effectiveness diminished when tasked

Agent Systems	Interactive Data Analysis & Dashboarding	Advanced Analytical Modelling	Data Engineering	Complex Multi-Source Integration	**Total**
Deepseek R1					
LLM Wrapper	3	1	1	1	**6**
Simple Agent System	4	2	0	1	**7**
Reviewing Agent System	3	3	2	1	**9**
Reviewing Agent System with lesser Critic	3	3	2	1	**9**
Llama 3.3 70B					
LLM Wrapper	1	1	0	0	**2**
Simple Agent System	2	1	0	0	**3**
Reviewing Agent System	2	1	0	0	**3**
Maximum possible Points					
	4	*4*	*2*	*4*	***14***

Table 5.2: Results of the Case Study using Top-k 3

with iterative refinement or modifications to existing dashboards. For instance, in Task 1, it struggled to correctly modify previously generated code in follow-up prompts, such as updating a dashboard to incorporate 'Experience in Current Domain'. This behaviur is likely attributable to the stateless nature of direct LLM API calls, where previously generated code or context is not inherently persistent unless explicitly re-provided in subsequent prompts.

The user experience with the LLM Wrapper was generally straightforward, and exhibited lower latency compared to the agent systems, the necessity of establishing a new testing environment for each prompt introduced practical overhead.

For Advanced Analytical Modelling, Deepseek R1 faced significant challenges. It failed to construct a functional neural network for attrition prediction in Task 1, even after multiple attempts and re-prompting with revised dataset details. This limitation naturally prevented it from progressing to utilize SHAP for model interpretability, which relied on a functional model.

In the domain of Data Engineering, the Deepseek R1 successfully performed the initial data cleaning steps in Task 3. However, it was unable to translate this processed data into an interactive frontend for visualizing the quantified data quality issues, indicating a gap in its ability to independently create user interfaces for data quality reports.

Regarding Complex Multi-Source Integration, Deepseek R1 could address the initial prompt of Task 4, which involved preliminary analysis of credit and loyalty card data. Nevertheless, it consistently faltered on subsequent prompts requiring the integration and analysis of multiple, more complex datasets. A notable observation was its attempt to use SQL for a prompt in Task 4, despite no database being referenced in the task, suggesting a potential misinterpretation or hallucination when confronted with intricate data

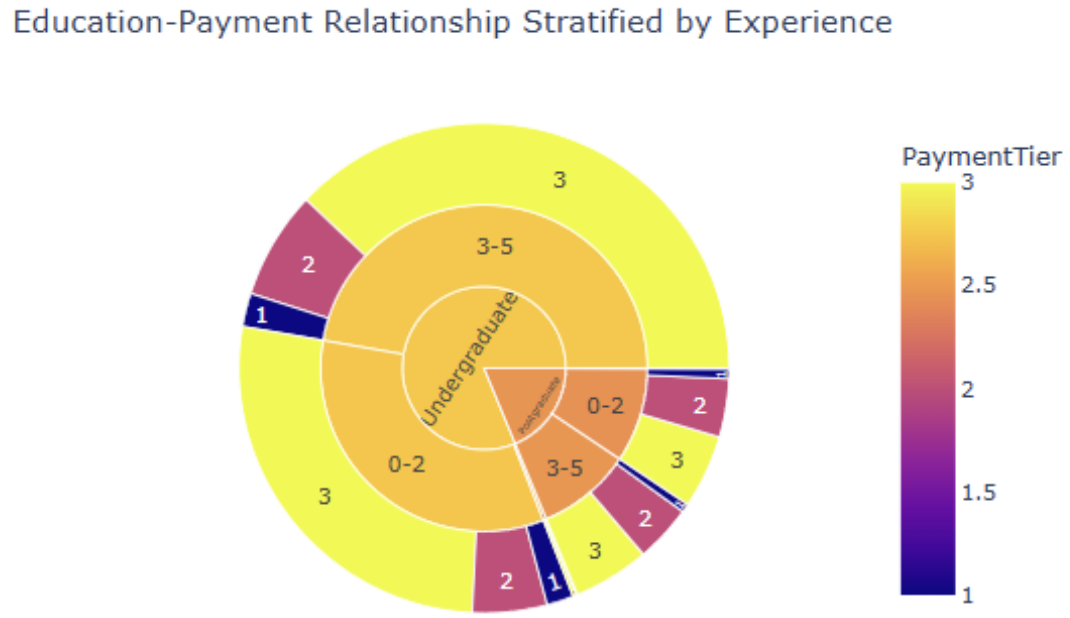


Figure 5.19: Graph generated by Simple Agent System using Deepseek R1

integration challenges.

In comparison, Llama 3.3 was only able to complete the third prompt of the first Task and the first prompt of the second Task. It was only able to complete the third prompt of the first Task after a fresh re-prompt, as it failed to create a working dashboard for the first prompt. In general, the model struggled to produce more than very simple dashboards.

5.4.2 Simple Agent System Performance

The Simple Agent System generally showed a marginal improvement in capabilities compared to the raw LLM Wrapper, particularly in its proactive approach to visualization.

In Interactive Data Analysis & Dashboarding, the Simple Agent System demonstrated a strong ability to generate initial graphs and dashboards when presented with data analysis tasks. This automatic visual output served as a valuable starting point for data exploration. For example, Deepseek R1 successfully solved the second prompt of Task 1, showing its capacity for dashboard updates. However, a consistent challenge observed across agent systems, including the Simple Agent System, was the design quality of LLM-generated visualizations. While often functionally correct, their visual clarity and user-friendliness could be compromised.

Figure 5.19, though not the sole example, highlights potential design issues in LLM-generated graphs. The concentric circles in this figure are intended to illustrate the relationship between education, payment tier, and experience. While the general utility of pie charts for conveying exact relations is debatable, they can offer a distinct method for visualizing hierarchical data structures. The centre section of the graph is intended to show that there is a substantially larger number of undergraduates (Bachelors) compared to postgraduates (Masters and PhD) in the dataset; however, the "postgraduates" label is rendered in such a small font that it becomes very difficult to read. Furthermore, the second circle from the centre represents years of experience at the company, but this

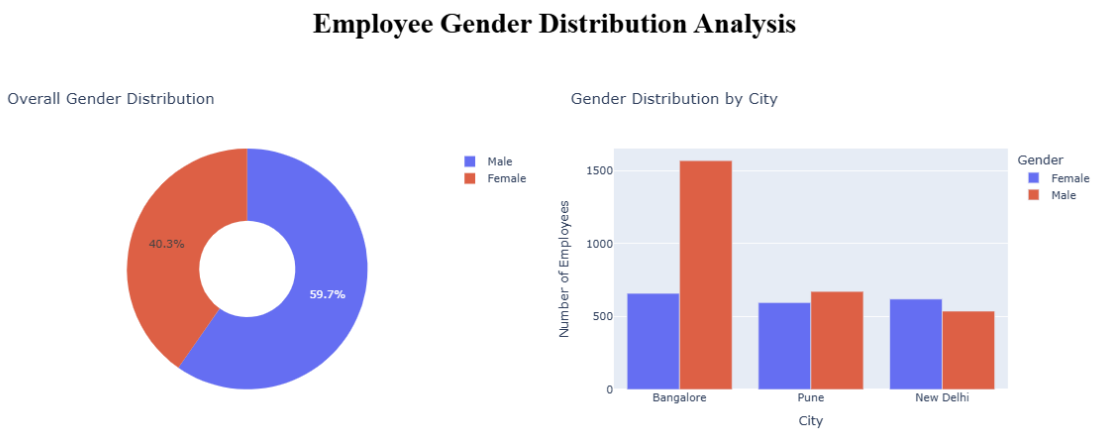


Figure 5.20: Graph generated by Simple Agent System using Deepseek R1

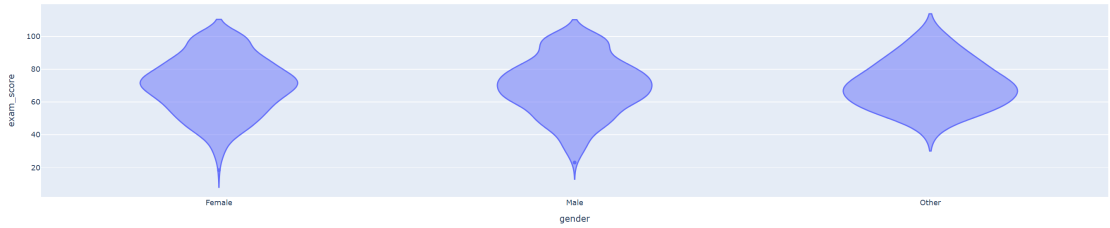


Figure 5.21: Graph generated by Simple Agent System using Llama 3.3

information is not explicitly stated in any accompanying labels. The interactive legend, accessible only by hovering over the circle in the frontend, also omits this data label. Experience is recorded as an integer value in the dataset, so the model's choice to categorize this into three groups appears reasonable. However, the rationale for the specific group boundaries is unclear. The dataset contains slightly over 4000 entries and is roughly split between employees with 0-2 years and 3+ years of experience. It is therefore difficult to ascertain why the model segregates the 17 entries with more than 5 years of experience into a distinct group, which is so small it is not readily visible in the static image. Nevertheless, this graph does indicate a weak or no clear relationship between experience and pay.

A subsequent visualization, shown in figure 5.20, presents a more problematic outcome by misleading the reader. In this figure, employees from Task 1 are categorized by gender. The graph on the left depicts the overall gender distribution (blue for male, red for female). The chart on the right shows gender distribution across different locations, but here the colour encoding is reversed (red for male, blue for female). No agent system, and neither model, is able to rectify these colour inconsistencies on their own consistently. This issue is not unique to the Simple Agent System.

Llama 3.3 can generate simple dashboards (such as the one in Figure 5.21, depicting the solved first prompt of Task 2) and successfully produces an additional dashboard using the Simple Agent System. Although it cannot handle more than three prompts, its latency is significantly lower than that of the reasoning model Deepseek R1. The model faced challenges of its own, as it often forgot to use the code tool, or announced the intended use of the tool, thereby calling it prematurely, and breaking the tool parser. This seems to occurs with less capable models; sporadic tests with other models (e.g., Gemini 2.0) showed similar issues.

Despite these visualization challenges, Deepseek R1 is able to implement a simple neural network. However, it subsequently fails to implement SHAP. In this instance, the agent attempts to rectify an error by adding an import statement without regenerating the complete, functional code segment, resulting in only the import statements being executed. After re-prompting, the agent is unable to reconstruct the full Dash implementation. Deepseek R1 also fails to implement the complex dashboard for the second task but is able to complete the dimensionality reduction component. It is unable to clean the smartwatch data of Task 3. For the fourth task, it is able to solve the first prompt. However, all agent systems, and models fail to correctly concatenate the GPS and transaction data in the subsequent prompt. While potentially not the optimal approach for this prompt, the agents encounter a technical issue, mismatching timestamp data formats (e.g., attempting to match 'm/d/Y' with 'd/m/Y'). This remains an issue even after re-prompting with the correct time format information.

5.4.3 Reviewing Agent Systems Performance

Both Reviewing Agent Systems (the standard Reviewing Agent System and Reviewing Agent System with lesser Critic) demonstrated very comparable and generally superior performance with Deepseek R1 compared to Llama 3.3.

In Interactive Data Analysis & Dashboarding, Deepseek R1 exhibited strong capabilities in generating functional dashboards and incorporating interactive elements. However, they consistently failed at the second prompt of Task 1, which required a comparative analysis involving education, experience, and pay. This specific challenge was due to a recurring import error related to the `statsmodels` library, which was not correctly added to the requirements list for the containerized execution environment.

For Advanced Analytical Modelling, Deepseek R1 showed significant advancements. It was able to successfully implement a simple neural network and effectively integrate SHAP for model interpretability, creating an interactive explanation component within the dashboard (see Figure 5.22). This highlights their enhanced ability to handle more complex analytical tasks and integrate advanced functionalities, although the visual refinement of the AI-generated frontends, while functional, still presented room for improvement.

In Data Engineering, Deepseek R1 successfully completed the data cleaning task for Task 3 and was capable of quantifying and visualizing data quality issues. Its ability to self-correct vague or incomplete responses, often termed *flake replies* [22], proved particularly beneficial in achieving robust data cleaning outcomes. Although Llama 3.3 attempted to clean the dataset and produced a purportedly cleaned one, a simple superficial inspection was enough to see that thsi was not really the case.

Regarding Complex Multi-Source Integration, similar to the other systems evaluated, the Reviewing Agent Systems with Deepseek R1 managed to solve the initial prompt of Task 4 but faced the same challenges with multi-source data concatenation and integration as observed with the Simple Agent System.

A practical observation common to both Reviewing Agent Systems, particularly when leveraging models with higher latency such as Deepseek R1, was the considerable time required for a full response. It was frequently observed that a functional dashboard would be generated, but the agent's accompanying textual explanation would be significantly delayed. This delay was more pronounced with Deepseek R1; when tasks were handled by the non-reasoning Llama 3.3 model, responses were quicker.

A subtle distinction between the two tested models was their proficiency in generating visually appealing textual answers; Deepseek R1 generally excelled in this regard,

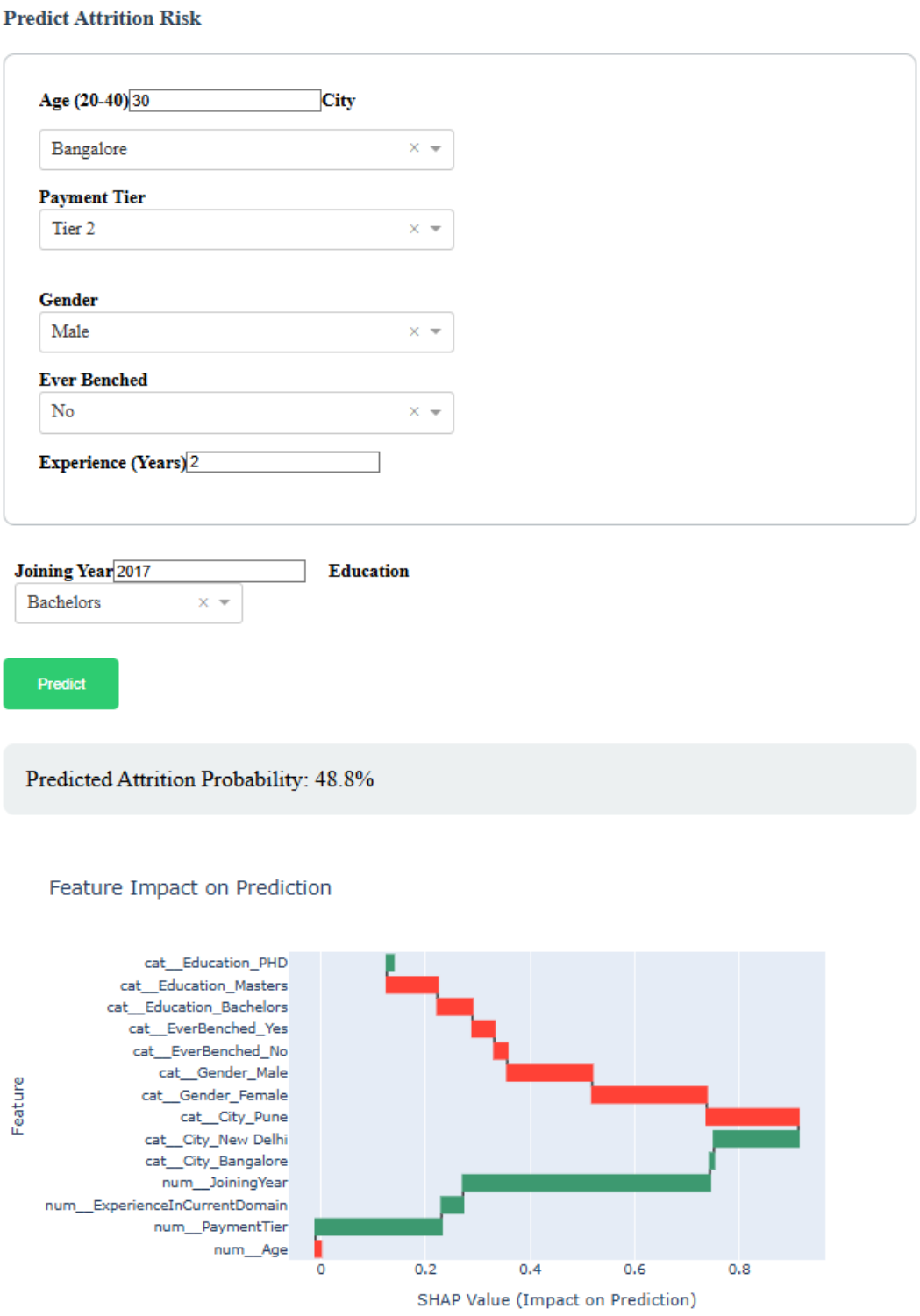


Figure 5.22: Interactive Visualization generated by Reviewing Agent System using Deepseek R1

whereas Llama 3.3, while providing effective textual summaries, tended to present actions narratively rather than using structured markdown results. This was not observed when Llama 3.3 was used as in the Simple Agent System or LLM Wrapper.

5.4.4 Planning Agent Systems Performance

The Planning Agent Systems exhibited significant usability challenges and generally underperformed. They displayed even greater latency than the Reviewing Agent Systems and frequently failed at relatively simple tasks. These systems often partitioned each prompt into excessively granular sub-tasks, leading to agents generating responses that resembled line-by-line task creation. The results of these sub-tasks were often incomplete and required correction in the final step. This approach frequently proved counterproductive, as the confident completion of individual sub-tasks did not consistently ensure the overall system functioned correctly. This could result in a final output where individual sub-tasks were marked as *confidently solved*, yet the overall task remained incomplete.

A further issue arose when the final step involved general directives such as *deploy the system* or *test the system*. This often led the models to initiate new, tangential project structures that impeded the completion of the original task. To investigate whether task granularity was a factor, modified prompts consolidating all prompts of a task into a single, larger prompt were also evaluated. This modification did not yield improved results; conversely, performance often degraded further, with the agent systems struggling to complete even basic tasks. Due to excessive execution time and consistently negative outcomes, the full suite of tasks, and models where not comprehensively tested on these systems.

6 Discussion

This thesis investigated the performance of LLM based agent systems equipped with diverse tools, memory mechanisms, and internal reasoning architectures for data analysis and code generation tasks. As LLMs increasingly demonstrate capabilities in streamlining complex data science processes, from data cleaning to model building and report writing, they are reshaping the traditional role of data scientists. This shift, as argued by Tu et al. (2024), moves data scientists from hands-on coding and analysis towards assessing and managing AI-performed analyses, akin to a product manager overseeing a development lifecycle [21]. The core aim of this thesis is to understand how different agentic frameworks compare to raw LLMs and simpler configurations within this evolving landscape, and to identify factors influencing their efficacy in supporting such data science workflows. This section discusses the principal findings, interprets their significance, and considers their implications.

6.1 RQ1: Agent System Efficacy and Architectural Impact

The study found that agent systems can, but do not universally, improve upon the performance of raw LLMs or simpler agent architectures like the LLM Wrapper. The specific internal reasoning architecture is a critical determinant of performance, which is also highly model-dependent. On the quantitative benchmark (Pass@1 metric, figure 5.14), more capable and especially reasoning models like o4 mini (high) and Deepseek R1 demonstrated performance gains when integrated into the Reviewing Agent System - e.g., o4 mini (high) improved from 58% as a Wrapper to 80% with the Reviewing Agent System . This suggests that for models with strong inherent reasoning, a structured review process can refine outputs and enhance task success. Conversely, highly proficient models like GPT 4.1 mini (80% as Wrapper) showed marginal deterioration within agent systems (78% for Simple and Reviewing). The Simple Agent System yielded mixed results; while it benefited some models compared to the Wrapper (e.g., o4 mini (high)), it proved detrimental for others, notably GPT 4.1 nano, whose performance plummeted from 69% (Wrapper) to 10% (Simple Agent System). This was hypothesized to be due to the model's tendency to generate conversational prefaces misinterpreted by the benchmark. The Planning Agent System and the Reviewing & Planning Agent System (figure 5.15), evaluated on GPT 4.1 mini and nano, consistently underperformed other configurations and incurred significant latency and computational costs, rendering them impractical for the tasks evaluated in this study. The case study (section 5.4) corroborated these findings, highlighting usability challenges and task completion failures for planning-based agents due to issues like excessive task granulation and flawed completion logic. This constitutes a negative result for the current planning design: over-decomposition, loss of context, and execution overhead outweighed any potential benefits. The initial hypothesis — that agent systems incorporating planning and/or reviewing would outperform raw models and simpler agents — can thus at best only be partially supported. . Reviewing agent systems did show benefits for certain models, planning agent systems did not.

The case study (section 5.4) further highlighted that beyond functional correctness, the usability and comprehensibility of AI-generated artifacts, such as dashboards, are important. LLM-generated visualizations in this study often suffered from design flaws (e.g., misleading color schemes, poor labeling, figure 5.19, 5.20), which agents did not consistently rectify. This difficulty in producing intuitively understandable and correctly formatted visualizations directly via LLM-generated code (Python for Dash in this instance) stands in contrast to strategies aimed at ensuring greater reliability for specific

visualization platforms. For example, the V-NLI system for Grafana proposed by Tong et al. (2024) deliberately separates natural language understanding from the generation of platform-specific JSON [19]. In their approach, the LLM interprets user requests and selects appropriate tools, but a custom, rule-based toolkit handles the precise JSON code generation needed for Grafana dashboards. This design choice is motivated by the observation that general-purpose LLMs can exhibit weaker capabilities in generating specialized, structured formats accurately. While such a hybrid approach might limit the expressive flexibility achievable through direct end-to-end code generation by an LLM, it aims to increase the reliability and correctness of the final visualization for a specific platform by relying on deterministic toolkits for the final rendering instructions. This contrasts with the broader, more flexible code generation attempted by the agents in this thesis, which, while versatile, demonstrated the aforementioned challenges in achieving consistently clear and well-designed visual outputs. This suggests a potential trade-off between generative flexibility and output reliability/quality in current LLM-agent systems for visualization tasks.

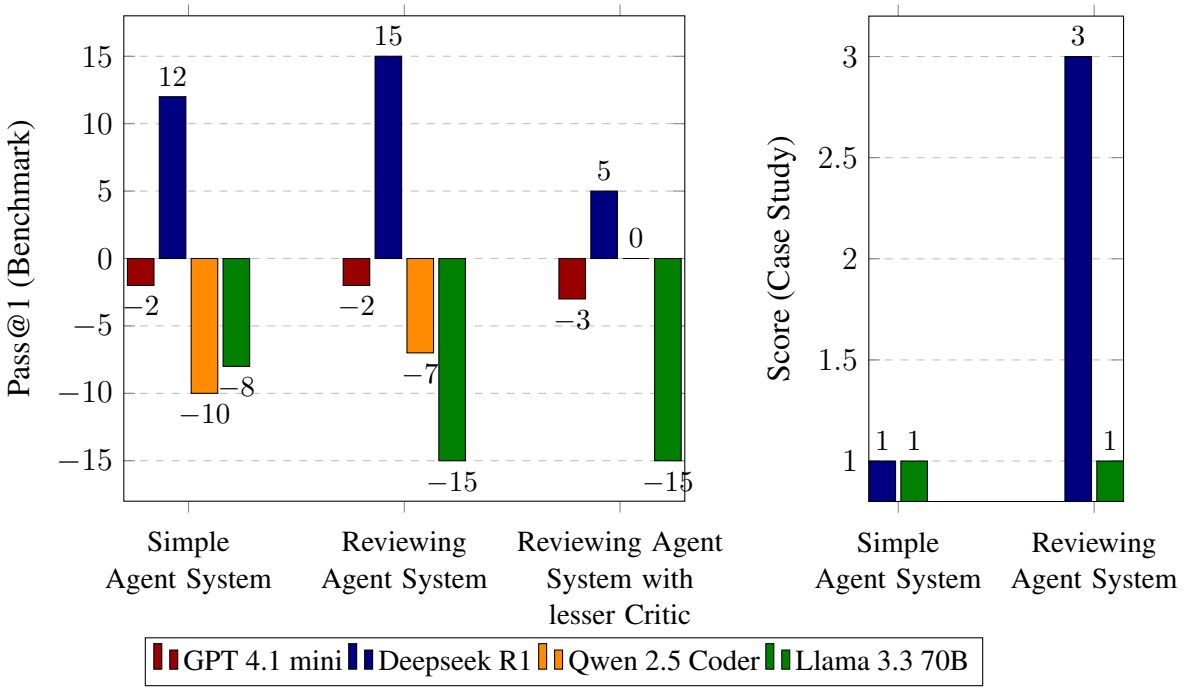


Figure 6.23: Relative improvements or deteriorations of LLMs using agent systems compared to an LLM Wrapper baseline (Top-k 3). Left: Benchmark; Right: Case Study.

6.2 RQ2: Impact of RAG Top-k for Coding Tutorials

The investigation into the optimal number of retrieved RAG documents (top-k, figures 5.17 and 5.18) revealed no universally ideal setting. Performance is highly sensitive to both the LLM and the agent architecture. For GPT 4.1 nano in the Simple Agent System, RAG context was generally detrimental, with k=0 being optimal. Surprising was also that GPT 4.1 nano had a worse benchmark score when given the results through the long-term memory feature in the Simple Agent System. In contrast, within the Reviewing Agent System, GPT 4.1 nano showed improved performance with moderate k values (k=1 to k=5). More capable models like Deepseek R1 (Simple Agent System) maintained high performance across various k values (0-100), peaking at k=1. GPT 4.1 mini similarly

remained robust across k with a mild optimum at k=10. GPT 4.1 mini was also able to improve its performance when given the results through the long-term memory feature in the Simple Agent System. This suggests that RAG, while intended to provide helpful context, can introduce noise or cognitive overload, particularly for smaller models or if the retrieval mechanism is not finely tuned to the task and model. The benefit of RAG is therefore not guaranteed and depends on careful configuration.

6.3 RQ3: Augmenting Smaller Models

The potential for structured agent systems to augment smaller or less capable models was also mixed. For GPT 4.1 nano, the Reviewing Agent (56% Pass@1) and particularly the Reviewing Agent with a "lesser critic" (Llama 3.3 70B, 66% Pass@1) did improve upon the Simple Agent's very low score (10%). However, these scores did not surpass its raw LLM Wrapper performance (69%). These results are mirrored in the results of other small models like Qwen 2.5 Coder with only 32B parameters. Though it has to be noted that Llama 3.3 70B has more parameters than Qwen 2.5 Coder 32B and potentially more than GPT 4.1 nano. Nevertheless both achieved higher scores in combination with Llama 3.3 than any of them achieved on their own in the Reviewing Agent System. If compared to reasoning models Deepseek R1 671B was able to improve its score more, in the benchmark, and the case study, compared to Llama 3.3 70B (see Figure 6.23). Compared to capable but non-reasoning models the result is more mixed. GPT 4.1 mini lost a couple of points in the benchmark with increasing agent system complexity. Its accompanying smaller model GPT 4.1 nano had quite substantial performance drops for the Simple Agent System, but was able to retrieve some points in the Reviewing Agent Systems. Although less aggressive Deepseek V3 671B performed comparable to GPT 4.1 nano, but more in line with Qwen 2.5 Coder. This indicates that while a structured agent framework can mitigate some failure modes of smaller models in basic agentic loops, it does not automatically elevate them to or beyond their standalone capabilities , as the case study demonstrates with Llama 3.3, only being able to generate basic dashboards across all execution platforms.

6.4 Additional Key Findings

The experiment with long-term memory seeded with ground truth solutions (figure 5.16) showed that some models (GPT 4.1 mini, Qwen 2.5 Coder) could leverage this "perfect" memory to improve Pass@1 scores significantly. However, other models (GPT 4.1 nano, Llama 4 Scout) exhibited performance degradation, underscoring that mere access to relevant information does not ensure its effective utilization. This complex reaction to readily available "correct" answers in the context is further illustrated by research on the faithfulness of Chain-of-Thought (CoT) reasoning in LLMs. Chen et al. (2024) [51] demonstrate that LLMs, when provided with explicit "hints" in their prompts — analogous to the ground truth solutions retrieved from long-term memory in this study — often use these hints to derive their final answer without explicitly acknowledging this reliance in their CoT. While their work focuses on the faithfulness of the reasoning trace rather than direct performance changes due to hints, it suggests that the internal mechanisms by which LLMs process and decide to use (or ignore, or misuse) such contextual cues are not always straightforward or transparent. The performance degradation observed for GPT 4.1 nano and Llama 4 Scout when long-term memory was active could, therefore, stem not merely from an inability to utilize the provided ground truth, but potentially from

complex, unstated internal reasoning paths where the "hint" is processed in a counterproductive manner or leads to the model forgoing more robust problem-solving strategies. Chen et al. also note that CoTs can sometimes be unfaithful even when models are not explicitly trained against a monitor, raising questions about the inherent transparency of LLM reasoning when provided with direct answers in context. This implies that even when perfect information is accessible via long-term memory, the agent's internal "decision" to use that information and how it influences the final output may not be evident from its explicit reasoning, and for some models, this interaction can be detrimental to task performance.

The case study further highlighted that beyond functional correctness, the usability and comprehensibility of AI-generated artifacts, such as dashboards, are important. LLM-generated visualizations often suffer from design flaws (e.g., misleading color schemes, poor labeling, figure 5.19, 5.20), which agents did not consistently rectify. Latency also emerged as a significant practical concern for more complex agent architectures like Reviewing and Planning systems.

6.5 Interpretation of Observations

The varied performance across different LLMs and agent architectures underscores the complex interplay between a model's inherent capabilities (e.g., reasoning, instruction following, susceptibility to distraction) and the structural constraints or aids provided by an agent system.

The success of the Reviewing Agent System for reasoning models like o4 mini (high) and Deepseek R1 likely stems from these models' ability to effectively process and act upon the critic's feedback, thereby refining their outputs. This structured interaction, where one agent (the tinker) performs the primary task and another (the critic) evaluates and guides, shares conceptual similarities with the role-playing framework proposed by Li et al. in CAMEL [22]. In CAMEL, an *AI user* agent provides instructions and an *AI assistant* agent executes them, aiming for autonomous cooperation. While the specific implementation details differ, both systems leverage a form of guided iteration and verification. The findings in this thesis that the Reviewing Agent System can enhance performance for certain models align with CAMEL's general premise that structured multi-turn interactions can lead to better task completion than single-shot generation. That the *lesser critic* sometimes yielding better results for certain models (e.g., GPT 4.1 nano, Qwen 2.5 Coder) is an intriguing finding. It might suggest that a critic doesn't always need to be a top-tier LLM; rather, its effectiveness could depend on its alignment with the primary model's output style or a reduced tendency to *over-correct* or introduce unhelpful complexity for smaller models. This concept of decomposing roles for better performance is also explored by frameworks like α-UMi [24], which uses separate, potentially smaller, LLMs for planning, tool calling, and summarization to improve tool learning, suggesting that specialized roles, even if not embodied by the most powerful LLMs, can be effective.

The drastic failure of GPT 4.1 nano in the Simple Agent System, and its sensitivity to RAG, points to a critical challenge: smaller or more conversational models, like Llama 3.3, may not be inherently suited for agentic loops that expect direct tool use or code output. The challenges CAMEL encountered, such as *flake replies* [22], also resonate with the difficulties observed when less constrained models interact within agentic frameworks.

The poor performance of the Planning Agent Systems suggests that the specific planning and task decomposition strategy implemented (section 4.6.2) may be suboptimal for

the evaluated tasks. The granular sub-tasks might have led to a loss of broader context or introduced opportunities for cumulative errors. This contrasts with approaches like Data Interpreter [6], which employs Hierarchical Graph modelling to dynamically structure and optimize data science workflows as interconnected task graphs, reportedly achieving significant performance improvements. Similarly, MetaGPT [14] aims to improve complex task execution, such as software development, by assigning agents distinct roles (e.g., Product Manager, Architect) that follow Standardized Operating Procedures (SOPs) and produce structured, standardized outputs. This highly structured, SOP-driven approach to decomposition and collaboration appears more robust than the more linear planning attempted in this thesis. Effective planning in LLM-agents thus remains a significant open research area, with graph-based methodologies and structured role-based collaboration presenting promising directions.

The qualitative findings of this thesis, particularly the generation of dashboards that are functionally correct but difficult to understand or even misleading (figures 5.19, 5.20), resonate strongly with the arguments presented by Tu et al. (2024). They emphasize the critical need for data scientists to develop robust critical thinking skills to evaluate LLM-generated outputs, which can suffer from "hallucinations" or make unsubstantiated assumptions. The observed UI design flaws are practical examples of why human oversight and critical assessment remain indispensable, even when tasks are largely automated by LLM-agents. If data scientists are to transition to a more managerial or strategic role as Tu et al. suggest, the reliability and interpretability of agent outputs become paramount [21].

The difficulties observed in this study with agents generating comprehensible and well-designed Dash UIs (section 5.4) also align with the broader challenges identified in the field of AI-assisted visual analytics. Bühler et al. (2024) emphasize that ensuring the integrity and transparency of AI-generated VA components remains a significant hurdle, with current LLMs still prone to limitations in generating consistently reliable and high-quality solutions for complex visualization tasks. The design flaws noted in figures 5.19 and 5.20 can be seen as manifestations of these ongoing challenges in moving from basic code generation to nuanced and effective visual communication by AI agents [17].

The RAG system implemented in this thesis (section 4.2) aims to ground agent responses and enhance code generation by providing access to a curated knowledge base of web documents, including programming tutorials and official documentation. This system employs OpenAI's `text-embedding-ada-002` and a ChromaDB vector store, with agents interacting via an automatic *Context Dump* (top-k controlled) and an explicit `<query>` tool. The results concerning RAG's impact (RQ2, section 5.3) indicated that its benefits are highly model- and architecture-dependent, with no universally optimal top-k setting, and that excessive retrieved information can be detrimental, especially for smaller models like GPT 4.1 nano. These findings highlight that successful RAG implementation involves not only effective retrieval but also judicious integration and utilization of the retrieved context by the LLM.

This approach aligns with the foundational principles of RAG as introduced by Lewis et al. [2], which combines pre-trained parametric memory (the LLM) with non-parametric memory (the retrieved documents) to improve performance on knowledge-intensive tasks. While the original RAG framework explored end-to-end training of retriever and generator components, this thesis focuses on using pre-trained retrievers and LLMs, with "tuning" occurring at the level of agent interaction strategy and top-k parameter selection. The observed sensitivity to top-k settings underscores the challenge that providing more information is only beneficial if the LLM can effectively discern relevance and integrate

it without being overwhelmed, a point also emphasized by Tu et al. regarding the general application of LLM knowledge [21].

Comparing this system to other contemporary RAG implementations reveals diverse strategies. For instance, DB-GPT [25] utilizes a more complex multi-source RAG system for database interactions, incorporating vector, keyword, and graph indexing from varied sources like PDFs and web pages, and focusing on privacy via private LLMs and Text-to-SQL fine-tuning. This contrasts with the primarily unstructured text and general coding/data science tutorial focus of the RAG in this thesis. The VotingAid system [28], while also employing a text-focused RAG with ChromaDB and OpenAI embeddings, applies it to a different domain — analyzing political party manifestos based on user-defined concerns rather than providing tutorial knowledge. This demonstrates RAG's versatility across different analytical objectives.

Regarding agent interaction with the RAG system, the "Context Dump" and explicit `<query>` tool in this thesis provide fixed mechanisms. AutoGen's "Retrieval-augmented Chat" [13] introduces a more dynamic "interactive retrieval" feature, where an LLM-agent can explicitly respond with "UPDATE CONTEXT" if initially retrieved documents are insufficient, thereby prompting further retrieval attempts. This offers a finer-grained agent control over the retrieval process than the automatic top-k dump.

These comparisons illustrate that while the core RAG principle is widely adopted, its implementation details — such as knowledge base structure, retrieval mechanisms (e.g., fixed top-k vs. similarity thresholds), indexing strategies, and the agent's interaction protocol with the RAG system — are crucial design choices. The findings of this thesis, particularly the model-dependent efficacy of RAG and the risks of information overload, contribute to the understanding that RAG is not a plug-and-play solution but requires careful integration and adaptation to the specific LLM, agent architecture, and task at hand.

6.6 Implications of the Study

This research has several implications for the development and application of LLM-based agent systems:

No One-Size-Fits-All Solution: The choice of LLM and agent architecture must be carefully tailored to the specific task and desired trade-offs between performance, cost, and latency. The varied successes of specialized multi-agent frameworks like MetaGPT [14] for software SOPs and α-UMi [24] for optimizing tool use in small LLMs further reinforce that tailored architectures outperform generic ones for specific complex domains. A sophisticated agent system is not a universal panacea and can be counterproductive if misaligned with the model's capabilities or the task's nature. This is crucial if agents are to reliably offload tasks from data scientists.

Value of Reviewing and Structured Collaboration: For tasks requiring high reliability, incorporating a reviewing or critic mechanism, as explored in this thesis, appears beneficial, particularly when paired with capable LLMs. This principle extends to more structured multi-agent collaborations, such as the role-based SOPs in MetaGPT [14] or the dynamic speaker selection and feedback loops enabled by frameworks like AutoGen [13], which aim to improve coherence and reduce errors. However, the nature and capability of the critic or collaborating agents warrant careful consideration.

RAG Configuration is Crucial: The "more data is better" assumption does not hold for RAG. Optimal top-k settings are highly variable, suggesting the need for dynamic or adaptive RAG strategies rather than fixed retrieval sizes, especially when dealing with diverse models.

Rethinking Planning Agents: The current implementation of planning agents proved inefficient. For complex, multi-step tasks, alternative approaches to planning and task decomposition, such as the graph-based modeling in Data Interpreter [6] or the SOP-driven task breakdown in MetaGPT [14], are needed to make such architectures viable for supporting strategic data science workflows.

Beyond Functional Correctness: Qualitative aspects like the comprehensibility of generated UIs and the overall user experience are important for practical adoption. Future agent development should incorporate principles of good design and clear communication in AI-generated artifacts.

Efficiency Considerations: The latency introduced by RAG, and complex agentic loops (especially planning and reviewing) is a significant barrier. For LLMs to truly shift the data scientist's focus to higher-level tasks, the underlying agent systems must be efficient and reliable.

7 Conclusion

This thesis presented the design, implementation, and comprehensive evaluation of a modular LLM-agent system featuring diverse reasoning architectures, memory mechanisms, and tool integration for data analysis and code generation tasks. The primary achievement was the agent sytem with four distinct agent architectures — Simple, Planning, Reviewing, and Planning & Reviewing. They where benchmarked against raw LLM performance and each other, utilizing both metrics (Pass@1 on parts of BigCodeBench) and qualitative assessments on multi-step interactive tasks, in the case study.

Key findings indicate that while agent systems can enhance LLM capabilities, their effectiveness is highly contingent on the specific agent architecture and the underlying LLM. The Reviewing Agent System demonstrated notable performance improvements for more capable reasoning models like o4 mini (high) and Deepseek R1, particularly in code generation, partially supporting the initial hypothesis that structured reasoning patterns would be beneficial. However, the Planning Agent System and its combination with reviewing, proved ineffective and inefficient for the evaluated tasks, contrary to initial expectations. The study also revealed that simpler agent structures can be detrimental to certain models, such as GPT 4.1 nano, highlighting a critical interaction between model characteristics and agent design. Furthermore, the research underscored that RAG requires careful, model-specific tuning of parameters like top-k, as more contextual information is not universally advantageous and can degrade performance, especially for smaller models. Similarly, the integration of long-term memory showed model-dependent utility. While agents could generate functional outputs like Dash dashboards, challenges in UI comprehensibility and design persist.

The importance of these results lies in providing empirical evidence on the practical trade-offs of different agent designs, offering guidance for developing more effective LLM-based tools for data analysis. The findings emphasize that a nuanced approach is necessary, moving beyond a general assumption that any agentic scaffolding will improve performance. Instead, the interaction between the LLM's inherent strengths, the task's nature, and the agent's structural overhead must be carefully considered. While specific performance figures are tied to the models and tasks herein, the observed principles regarding architectural impact, RAG sensitivity, and model-agent alignment are likely generalizable to other agent systems and data-centric applications.

The principal limitations of this study include the evaluation of a specific subset of available LLMs, the use of a static RAG knowledge base, and the reliance on simulated tasks without live user studies for qualitative assessment. The evaluation of the case study was also conducted by a single researcher. These factors define the boundaries within which the current conclusions are drawn.

8 Future Works

The findings and limitations of this research open several avenues for future work, aiming to enhance the capabilities, reliability, and practicality of LLM-based agent systems.

Refinements to the current system could focus on improving agent-environment interaction and code management. Future iterations could grant agents more direct control over their execution environment, which might align more naturally with their code generation patterns. Enhancing the agents' capability to iteratively modify and add to existing code within a persistent codebase, perhaps adopting a Git-like versioning or diffing mechanism, would address observed limitations in context retention for follow-up tasks and foster more realistic development workflows. Furthermore, enabling users to directly edit agent-generated code within the interface would significantly improve the collaborative aspect, allowing for quick human-led corrections and refinements.

The RAG component also presents opportunities for significant advancement. Instead of relying solely on a fixed top-k for document retrieval, RAG systems could be adapted to use dynamic similarity thresholds, potentially offering more contextually relevant information. Introducing scenario-specific rules or structured knowledge into the RAG, for example, providing guidelines for colour use in visualizations or standardized pipelines for handling common data types like datetimes, could improve the quality and consistency of generated outputs.

The evaluation methodology for agent-generated artifacts, particularly user interfaces, warrants expansion. Future automated visual evaluation could incorporate interactivity, allowing the assessment system to click buttons, navigate through generated dashboards, and thereby identify usability issues or hidden flaws not apparent from static analysis.

Further research should also explore broader architectural and model variations. Testing with a wider range of LLMs, including those specialized for different tasks or with different architectural underpinnings, would help generalize the findings on model-agent interactions. The intriguing observation that a "lesser critic" sometimes benefited certain models in the Reviewing Agent System merits deeper investigation into the optimal characteristics and prompting of critic modules. Exploring hybrid agent architectures that can dynamically adapt their reasoning strategy (e.g., switching from a Simple to a Reviewing loop based on task complexity or observed errors) could lead to more robust and efficient systems.

Finally, conducting live user studies is crucial for validating the practical utility and user experience of these agent systems. Understanding how users with varying levels of technical expertise interact with these tools, perceive their outputs ("vibes"), and integrate them into their workflows will provide invaluable feedback for designing truly effective and user-centric AI-powered data analysis assistants.

Beyond technical evaluations, future work could also focus on translating these findings into actionable insights for organizations and decision-makers, bridging the gap between technical innovation and practical adoption in business intelligence workflows.

Addressing these unanswered questions and pursuing these improvements can lead to more powerful and reliable LLM-agents with tangible real-world applications in data science and beyond.

9 Web References

The primary software components and resources developed or utilized in this thesis are publicly accessible. The backend, frontend, and database management modules for the AI Agent system, along with the scripts and data for the benchmark and case study, are available in their respective GitHub repositories. A deployed version of the AI Agent tool is also accessible online.

- AI Agent System Backend: github.com/NicolasMahn/AI_Agents_Backend
- AI Agent System Frontend: github.com/NicolasMahn/AI_Agents_Frontend
- AI Agent System Database Manager: github.com/NicolasMahn/AI_Agent_DB_Manager
- Benchmark Resources: github.com/NicolasMahn/QuantitativeAgentBenchmark
- Case Study Resources: github.com/NicolasMahn/QualitativeAgentAnalysis

References

[1] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul, S. Brunner, C. Gong, T. Hoang, A. R. Zebaze, X. Hong, W.-D. Li, J. Kaddour, M. Xu, Z. Zhang, P. Yadav, N. Jain, A. Gu, Z. Cheng, J. Liu, Q. Liu, Z. Wang, B. Hui, N. Muennighoff, D. Lo, D. Fried, X. Du, H. de Vries, and L. V. Werra, "BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions," 2025. [Online]. Available: https://arxiv.org/abs/2406.15877

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," 2021. [Online]. Available: https://arxiv.org/abs/2005.11401

[3] Z. Liu, W. Yao, J. Zhang, L. Yang, Z. Liu, J. Tan, P. K. Choubey, T. Lan, J. Wu, H. Wang, S. Heinecke, C. Xiong, and S. Savarese, "AgentLite: A lightweight library for building and advancing task-oriented LLM agent system," 2024. [Online]. Available: https://arxiv.org/abs/2402.15538

[4] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, "Executable code actions elicit better LLM agents," in *Forty-first International Conference on Machine Learning*, 2024. [Online]. Available: https://openreview.net/forum?id=jJ9BoXAfFa

[5] Anthropic *et al.*, "Model context protocol," 2024, accessed: 2025-09-08. [Online]. Available: https://modelcontextprotocol.io/

[6] S. Hong, Y. Lin, B. Liu, B. Liu, B. Wu, C. Zhang, C. Wei, D. Li, J. Chen, J. Zhang, J. Wang, L. Zhang, L. Zhang, M. Yang, M. Zhuge, T. Guo, T. Zhou, W. Tao, X. Tang, X. Lu, X. Zheng, X. Liang, Y. Fei, Y. Cheng, Z. Gou, Z. Xu, and C. Wu, "Data Interpreter: An LLM agent for data science," 2024. [Online]. Available: https://arxiv.org/abs/2402.18679

[7] S. Guo, C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang, "DS-Agent: Automated data science by empowering large language models with case-based reasoning," 2024. [Online]. Available: https://arxiv.org/abs/2402.17453

[8] Y. Talebirad and A. Nadiri, "Multi-agent collaboration: Harnessing the power of intelligent LLM agents," 2023. [Online]. Available: https://arxiv.org/abs/2306.03314

[9] S. Wu, S. Zhao, Q. Huang, K. Huang, M. Yasunaga, K. Cao, V. N. Ioannidis, K. Subbian, J. Leskovec, and J. Zou, "Avatar: Optimizing llm agents for tool usage via contrastive reasoning," 2024. [Online]. Available: https://arxiv.org/abs/2406.11200

[10] OpenAI. (2024) Models - GPT-4o - OpenAI API. Accessed on May 28, 2025. The year 2024 is assumed based on the model's release date; the documentation is continuously updated. [Online]. Available: https://platform.openai.com/docs/models/chatgpt-4o-latest

[11] Artificial Analysis, "AI model leaderboard," https://artificialanalysis.ai/leaderboards/models, Apr. 2025, accessed: 2025-04-22. [Online]. Available: https://artificialanalysis.ai/leaderboards/models

[12] S. L. Nikiema, J. Samhi, A. K. Kaboré, J. Klein, and T. F. Bissyandé, "The code barrier: What llms actually understand?" 2025. [Online]. Available: https://arxiv.org/abs/2504.10557

[13] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen llm applications via multi-agent conversation," 2023, version 2. Code available at https://github.com/microsoft/autogen. [Online]. Available: https://arxiv.org/abs/2308.08155

[14] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "Metagpt: Meta programming for a multi-agent collaborative framework," 2024, version 2. Code available at https://github.com/geekan/MetaGPT. [Online]. Available: https://arxiv.org/abs/2308.00352

[15] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. [Online]. Available: http://www.jstor.org/stable/25148625

[16] A. Backlund and L. Petersson, "Vending-bench: A benchmark for long-term coherence of autonomous agents," 2025. [Online]. Available: https://arxiv.org/abs/2502.15840

[17] K. Bühler, T. Höllt, T. Schulz, and P.-P. Vázquez, "Ai-in-the-loop: The future of biomedical visual analytics applications in the era of ai," 2024. [Online]. Available: https://arxiv.org/abs/2412.15876

[18] "Cost to run artificial analysis intelligence index," https://artificialanalysis.ai/#cost-to-run-artificial-analysis-intelligence-index, accessed: 2025-09-08.

[19] B.-Y. Tong, T. T. Kuo, and C.-Y. Lin, "Visualization-oriented natural language interfaces for grafana dashboard," in *2024 International Conference on Consumer Electronics - Taiwan (ICCE-Taiwan)*, 2024, pp. 465–466.

[20] "LangChain," https://www.langchain.com/, LangChain, Apr. 2025, accessed: 2025-04-22. [Online]. Available: https://www.langchain.com/

[21] X. Tu, J. Zou, W. J. Su, and L. Zhang, "What should data science education do with large language models?" 2023. [Online]. Available: https://arxiv.org/abs/2307.02792

[22] G. Li, H. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 51 991–52 008. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/file/a3621ee907def47c1b952ade25c67698-Paper-Conference.pdf

[23] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, "A dynamic LLM-powered agent network for task-oriented agent collaboration," 2024. [Online]. Available: https://arxiv.org/abs/2310.02170

[24] W. Shen, C. Li, H. Chen, M. Yan, X. Quan, H. Chen, J. Zhang, and F. Huang, "Small LLMs are weak tool learners: A multi-LLM agent," 2024. [Online]. Available: https://arxiv.org/abs/2401.07324

[25] S. Xue, C. Jiang, W. Shi, F. Cheng, K. Chen, H. Yang, Z. Zhang, J. He, H. Zhang, G. Wei, W. Zhao, F. Zhou, D. Qi, H. Yi, S. Liu, and F. Chen, "DB-GPT: Empowering database interactions with private large language models," 2024. [Online]. Available: https://arxiv.org/abs/2312.17449

[26] R. Tavares and B. Tavares, "Beyond the veil of secrecy: Unveiling the enigma of long covid through a novel therapeutic approach," *Research Square*, apr 2024. [Online]. Available: https://assets-eu.researchsquare.com/files/rs-4215447/v1_covered_061b6e2f-4206-486f-8722-08d93b71cafe.pdf

[27] C. Zong, Y. Yan, W. Lu, J. Shao, E. Huang, H. Chang, and Y. Zhuang, "Triad: A framework leveraging a multi-role LLM-based agent to solve knowledge base question answering," 2024. [Online]. Available: https://arxiv.org/abs/2402.14320

[28] N. Mahn and N. Abbas, "VotingAid: Artificial Intelligence (AI) based Retrieval-Augmented Generation LLM Approach for Political Decision-Making," in *CEUR Workshop Proceedings (CEUR-WS.org)*, 2025, just Accepted.

[29] Y. Wen, P. Yin, K. Shi, H. Michalewski, S. Chaudhuri, and A. Polozov, "Grounding data science code generation with input-output specifications," 2024. [Online]. Available: https://arxiv.org/abs/2402.08073

[30] Y. Pan, C. Lyu, Z. Yang, L. Li, Q. Liu, and X. Shao, "E-code: Mastering efficient code generation through pretrained models and expert encoder group," 2024. [Online]. Available: https://arxiv.org/abs/2408.12948

[31] C. Improta, P. Liguori, R. Natella, B. Cukic, and D. Cotroneo, "Enhancing robustness of ai offensive code generators via data augmentation," *Empirical Software Engineering*, vol. 30, no. 1, Oct. 2024. [Online]. Available: http://dx.doi.org/10.1007/s10664-024-10569-y

[32] S. Wu, S. Zhao, M. Yasunaga, K. Huang, K. Cao, Q. Huang, V. N. Ioannidis, K. Subbian, J. Zou, and J. Leskovec, "Stark: Benchmarking llm retrieval on textual and relational knowledge bases," in *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, Eds., vol. 37. Curran Associates, Inc., 2024, pp. 127 129–127 153. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/file/e607b1419e9ae7cd5cb5b5bb60c2ad5c-Paper-Datasets_and_Benchmarks_Track.pdf

[33] "DeepSeek AI," https://huggingface.co/deepseek-ai, DeepSeek AI, Apr. 2025, hugging Face organization page. Accessed: 2025-04-22. [Online]. Available: https://huggingface.co/deepseek-ai

[34] BigCode, "BigCode models leaderboard," https://huggingface.co/spaces/bigcode/bigcode-models-leaderboard, Apr. 2025, accessed: 2025-04-22. Hosted on Hugging Face Spaces. [Online]. Available: https://huggingface.co/spaces/bigcode/bigcode-models-leaderboard

[35] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Morikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[36] R. K. Yin, "Case study research," 2003.

[37] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009. [Online]. Available: https://doi.org/10.1007/s10664-008-9102-8

[38] T. Elmetwally, "Employee dataset," https://www.kaggle.com/datasets/tawfikelmetwally/employee-dataset, accessed: 2025-04-25.

[39] S. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," 2017. [Online]. Available: https://arxiv.org/abs/1705.07874

[40] J. A. Naath, "Student habits vs academic performance," https://www.kaggle.com/datasets/jayaantanaath/student-habits-vs-academic-performance, accessed: 2025-04-25.

[41] M. A. R, "Smartwatch health data - uncleaned," https://www.kaggle.com/datasets/mohammedarfathr/smartwatch-health-data-uncleaned, accessed: 2025-04-25.

[42] VAST Challenge, "VAST Challenge 2021: Mini-Challenge 2," https://vast-challenge.github.io/2021/MC2.html, 2021, accessed: June 7, 2025.

[43] M. Renze and E. Guven, "The effect of sampling temperature on problem solving in large language models," 2024. [Online]. Available: https://arxiv.org/abs/2402.05201

[44] F. Lin, D. J. Kim, Tse-Husn, and Chen, "Soen-101: Code generation by emulating software process models using large language model agents," 2024. [Online]. Available: https://arxiv.org/abs/2403.15852

[45] Z. Lu, Q. Jin, J. Xi, Q. Chen, Y. Zhang, X. Zhang, Y. Yang, and J. Cheng, "Comparison of large language model performance in answering questions about rare diseases: ChatGPT-3.5, ChatGPT-4o, Gemini-1.5-pro, and Claude-3-opus," *medRxiv*, 2024, preprint posted July 23, 2024. [Online]. Available: https://www.medrxiv.org/content/10.1101/2024.07.22.24310824v1

[46] OpenAI, “Models - OpenAI API,” https://platform.openai.com/docs/models, Apr. 2025, accessed: 2025-04-22. [Online]. Available: https://platform.openai.com/docs/models

[47] “Meta Llama,” https://huggingface.co/meta-llama, Meta, Apr. 2025, hugging Face organization page. Accessed: 2025-04-22. [Online]. Available: https://huggingface.co/meta-llama

[48] “Models,” https://ai.google.dev/gemini-api/docs/models, Google, Apr. 2025, google AI for Developers documentation. Accessed: 2025-04-22. [Online]. Available: https://ai.google.dev/gemini-api/docs/models

[49] Y. Ning, L. Hai, L. Yibo, T. Baoliang, L. Guoqing, and Z. Haijun, “Token-importance guided direct preference optimization,” 2025. [Online]. Available: https://arxiv.org/abs/2505.19653

[50] X. Wu, Y. Wang, S. Jegelka, and A. Jadbabaie, “On the emergence of position bias in transformers,” 2025. [Online]. Available: https://arxiv.org/abs/2502.01951

[51] Y. Chen, J. Benton, A. Radhakrishnan, J. Uesato, C. Denison, J. Schulman, A. Somani, P. Hase, M. Wagner, F. Roger, V. Mikulik, S. Bowman, J. Leike, J. Kaplan, and E. Perez, “Reasoning Models Don’t Always Say What They Think,” 2025, Alignment Science Team, Anthropic. [Online]. Available: https://assets.anthropic.com/m/71876fabef0f0ed4/original/reasoning_models_paper.pdf

A Further Information about the Case Study

A.1 Prompts used for the Case Study

Task 1 Prompts:

1. I have compiled a dataset of employees at my company with the following columns:

    1. **Education:** The educational qualifications of employees, including degree, institution, and field of study.
    2. **Joining Year:** The year each employee joined the company, indicating their length of service.
    3. **City:** The location or city where each employee is based or works.
    4. **Payment Tier:** Categorization of employees into different salary tiers.
    5. **Age:** The age of each employee, providing demographic insights.
    6. **Gender:** Gender identity of employees, promoting diversity analysis.
    7. **Ever Benched:** Indicates if an employee has ever been temporarily without assigned work.
    8. **Experience in Current Domain:** The number of years of experience employees have in their current field.
    9. **Leave or Not:** Employees that left the company.

    I want to analyze the relationship between employee education and their payment tier.

    1. How does the level of education relate to the Payment Tier? (Assume higher degrees like Master's or PhD represent 'better' education compared to Bachelor's. Please state any assumptions made about categorizing education levels).
    2. Create a Plotly Dash dashboard to visualize this relationship. Consider using visualizations like box plots showing the distribution of Payment Tiers for each education category, or a bar chart showing the average/median Payment Tier per education category.

2. Building on the previous analysis:

    1. How does 'Experience in Current Domain' relate to the 'Payment Tier'?
    2. Re-examine the relationship between 'Education' and 'Payment Tier' while statistically controlling for the influence of 'Experience in Current Domain'.
    3. Update the existing Plotly Dash dashboard:

        - Add a new visualization showing the relationship between 'Experience in Current Domain' and 'Payment Tier'.
        - Add another visualization (or modify the existing education vs. payment tier chart) to show the relationship between 'Education' and 'Payment Tier' *after* accounting for experience levels.

3. Create a *new* dashboard using the employee dataset to analyze gender distribution.

    1. What is the overall gender distribution within the workforce?

2. How does the gender distribution vary across different 'City' locations?
3. Create a Plotly Dash dashboard visualizing these findings. Use appropriate charts, such as a pie chart for the overall distribution and grouped or stacked bar charts to compare distributions across cities.

4. Create a *new* dashboard to analyze and predict employee attrition ('Leave or Not').

    1. Explore potential patterns in leave-taking behavior. Specifically, investigate correlations between 'Leave or Not' and variables like 'Age', 'Payment Tier', 'Experience in Current Domain', 'Ever Benched', and 'Gender'. Visualize significant correlations.
    2. Build a predictive model (e.g., Logistic Regression, suggest alternatives if more appropriate) to estimate the probability of an employee leaving based on their attributes.
    3. Create a Plotly Dash dashboard that includes:

        - Visualizations of the key patterns identified in step 1.
        - An interface where a user can input relevant employee details (specify which ones: e.g., Age, City, Payment Tier, Gender, Ever Benched, Experience, Joining Year, Education).
        - A display showing the model's predicted probability of that employee leaving.

5. Update the employee attrition prediction dashboard created in the previous step.

    1. Add a component (e.g., a chart or table) that helps interpret the prediction for the *currently inputted employee*. Show *local* feature importance, highlighting which input features contributed most significantly (positively or negatively) to the predicted probability of leaving for that specific case (e.g., using SHAP or LIME).
    2. Integrate this explanation clearly alongside the prediction output in the dashboard interface.

Task 2 Prompts:

1. Using the provided 'student_habits_performance.csv' dataset:

    1. How do the exam scores (`exam_score`) of female students compare to male students, based on the `gender` column (e.g., compare average/median scores)?
    2. How do the distributions of `exam_score` differ between genders?
    3. Create a Plotly Dash dashboard to visualize these comparisons. Use visualizations like side-by-side box plots to show `exam_score` distributions per `gender` and potentially a bar chart comparing average `exam_score`.

2. Create a *new* dashboard for exploring the 'student_habits_performance.csv' dataset using dimensionality reduction (DR).

    1. Select relevant features for DR.

2. Perform dimensionality reduction (reducing to 2 dimensions for visualization) on the preprocessed features.
3. Allow the user to select the DR method from a list of common options (e.g., PCA, t-SNE, UMAP).
4. Create a Plotly Dash dashboard featuring an interactive 2D scatter plot visualizing the results of the chosen DR method. Allow points to be colored initially by `gender`.

3. Update the dimensionality reduction dashboard created previously (using 'student_habits_performanc to include clustering capabilities.

    1. Add functionality to perform clustering. Clustering is to be done on the original preprocessed features.
    2. Allow the user to select a clustering algorithm from a list (e.g., K-Means, DBSCAN, Agglomerative Clustering) and input necessary parameters (e.g., number of clusters 'k' for K-Means).
    3. Update the interactive 2D scatter plot (which shows the dimensionally reduced data) so that the user can choose how the points are colored:

        - Color by the cluster labels generated by the selected clustering algorithm applied to the original features.
        - Color by the values of any other selected attribute from the original dataset (e.g., `gender`, `part_time_job`, `diet_quality`, `internet_quality`, `parental_education_level`).

Task 3 Prompts:

1. Using the provided 'unclean_smartwatch_health_data.csv' dataset, which contains missing values, potential outliers, duplicates, and inconsistencies as observed in the schema and preview:

    1. **Clean the data.** Please perform and document the following cleaning steps, stating the strategy chosen where applicable:

        - Handle Missing Values
        - Correct Data Types
        - Standardize Categorical Values
        - Identify & Handle Outliers

    2. **Create a new cleaned data file** named `cleaned_smartwatch_health_data.csv` containing the cleaned data.

2. Using the *original* `unclean_smartwatch_health_data.csv` dataset:

    1. Analyze and Quantify Data Quality Issues per Column
    2. **Create a Plotly Dash dashboard** that visualizes these quantified issues. Use clear charts (e.g., bar charts) to show the count of missing values, defined outliers, identified type errors, and non-standard categories for each relevant column. Include a summary count of total duplicate rows.

Task 4 The data sources for this task contain the following files:

- *car-assignment.csv*: A table containing the names of the employees and what car they are driving
- *gps.csv*: Holds the coordinates of the cars.
- *cc_data.csv*: Contains the transaction history of the credit cards.
- *loyalty_data.csv*: Contains the transaction history of the loyalty cards.
- *aliba.json*: A Geojson of the street-network.
- *tourist_map.jpg*: A tourist map of the area.
- *README.md*: A short description of all files

Prompts:

1. Create a dashboard using just the credit and loyalty card data, identify the most popular locations, and when they are popular. What anomalies do you see? What corrections would you recommend to correct these anomalies? Create a plotly dash frontend that visualizes this.
2. Add the vehicle data to your analysis of the credit and loyalty card data. How does your assessment of the anomalies in question 1 change based on this new data? What discrepancies between vehicle, credit, and loyalty card data do you find? Update the plotly dash frontend with the answer.
3. Can you infer the owners of each credit card and loyalty card? What is your evidence? Where are there uncertainties in your method? Where are there uncertainties in the data? Update the plotly dash frontend with the answer.
4. Given the data sources provided, identify potential informal or unofficial relationships among GASTech personnel. Provide evidence for these relationships. Update the plotly dash frontend with the answer.

A.2 Overarching Evaluation Criteria for Case Study Tasks

1. **Interactive Data Analysis & Dashboarding:**

   - **Focus:** The agent's ability to perform iterative data exploration, generate descriptive statistics, create a variety of informative visualizations (e.g., bar charts, box plots, scatter plots), and build interactive Plotly Dash dashboards that allow for user interaction and dynamic updates.
   - **Applicability:** Directly relevant to **Task 1** (initial data exploration, comparison of education/payment tier, gender distribution) and **Task 2** (gender-based exam score comparison).

2. **Advanced Analytical Modeling:**

- **Focus:** The agent's capability to implement more sophisticated analytical techniques, including machine learning models for prediction (e.g., neural networks, logistic regression), and to integrate model interpretability methods (e.g., SHAP, LIME) within interactive applications to provide actionable insights.
- **Applicability:** Primarily evaluated in **Task 1** (neural network for attrition prediction, SHAP integration for local feature importance) and **Task 2** (dimensionality reduction visualizations, clustering visualizations).

3. **Data Engineering:**

   - **Focus:** The agent's proficiency in handling real-world data challenges, including comprehensive data preprocessing steps (e.g., missing value imputation, type correction, outlier handling, standardization), and the ability to quantify and visualize data quality issues.
   - **Applicability:** Directly assessed in **Task 3** (cleaning the smartwatch health data, quantifying and visualizing data quality issues).

4. **Complex Multi-Source Integration:**

   - **Focus:** The agent's capacity to integrate and analyze information from multiple, diverse data sources (e.g., tabular data, geospatial data, images), identify complex relationships and anomalies across these sources, and manage a larger, multi-step analytical project with potentially ambiguous objectives. This also probes the system's scalability and limits.
   - **Applicability:** Central to **Task 4** (integrating credit/loyalty card, vehicle, GPS, and street network data to identify patterns, anomalies, and relationships).

B Example Prompt and Simple Agent System Solution in Quantitative Benchmark

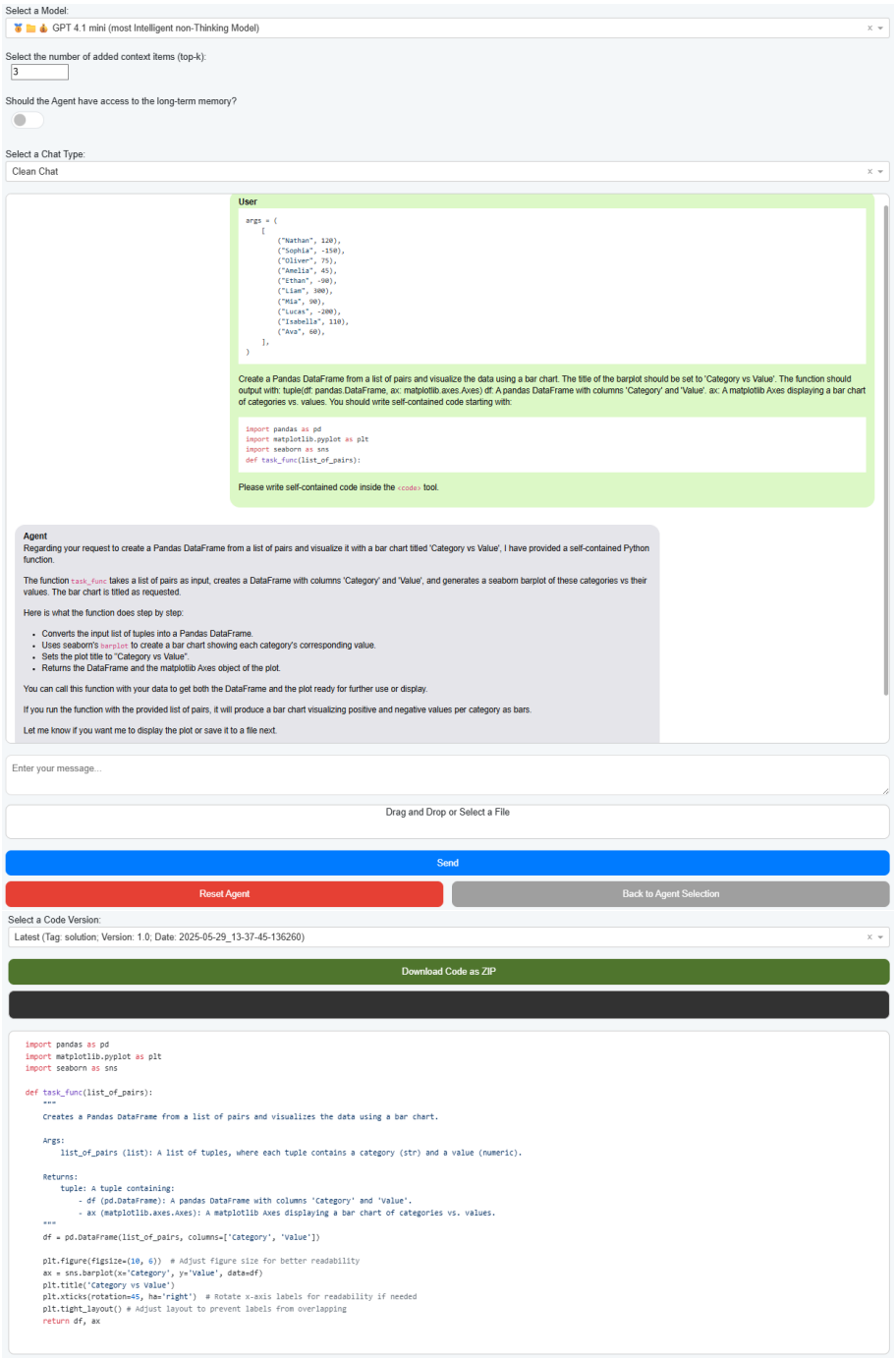


Figure B.24: Example Quantitative Benchmark with GPT 4.1 mini and Top-k 3